SID4: 1066

PORT_BASE: 8166

PREFIX: s1066

SEED: 1066

VERIFY_SEED: 261066

DOMAIN_ID: 2

Hardware: Macintosh HD, Apple M4, 24 GB, Tahoe 26.6.2

Local Model Used: Ollama qwen3:8b

Tagged Commit Hash:

https://github.com/paknathan/data260-1066

Part 1) Congestion Index is the additional entity that will relate to Municipal Transit Incident. Congestion Index measures the level of congestion where the incident occurred.

I & II) Database & API Implementation

Main.py

```python
DATABASE_URL = os.getenv("DATABASE_URL")

if not DATABASE_URL:
    raise RuntimeError("DATABASE_URL environment variable is not set. Run `export DATABASE_URL=...` first.")

engine = create_engine(DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

# --- Auth Models ---
class UserModel(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    name = Column(String(100), nullable=False)
    email = Column(String(120), unique=True, index=True, nullable=False)
    password_hash = Column(String(255), nullable=False)

    sessions = relationship("SessionModel", back_populates="user", cascade="all, delete-orphan")

class SessionModel(Base):
    __tablename__ = "sessions"

    id = Column(String(64), primary_key=True, index=True)
    user_id = Column(Integer, ForeignKey("users.id"), nullable=False)
    created_at = Column(DateTime, default=datetime.utcnow, nullable=False)
    expires_at = Column(DateTime, nullable=False)

    user = relationship("UserModel", back_populates="sessions")

# --- HW5 Secondary Entity Table ---
class CongestionIndexModel(Base):
    __tablename__ = "Congestion_indexes"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    level_name = Column(String(100), nullable=False)          # Primary text field
    description = Column(Text, nullable=True)                 # Secondary text field
    index_code = Column(String(20), unique=True, nullable=False, index=True) # Unique field
    created_at = Column(DateTime, server_default=func.now())
    updated_at = Column(DateTime, server_default=func.now(), onupdate=func.now())

    incidents = relationship("IncidentModel", back_populates="congestion_index")

# --- HW5 Updated Primary Entity Table ---
class IncidentModel(Base):
    __tablename__ = "incidents"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    title = Column(String(255), nullable=False)               # Primary text field
    incident_code = Column(String(50), unique=True, nullable=False, index=True) # Unique field
```

```python
    incident_code = Column(String(50), unique=True, nullable=False, index=True) # Unique field
    delay_minutes = Column(Integer, nullable=False, default=0) # Numeric field with default
    congestion_index_id = Column(Integer, ForeignKey("Congestion_indexes.id", ondelete="RESTRICT"), nullable=False)
    created_at = Column(DateTime, server_default=func.now())
    updated_at = Column(DateTime, server_default=func.now(), onupdate=func.now())

    congestion_index = relationship("CongestionIndexModel", back_populates="incidents")
    logs = relationship("IncidentLogModel", back_populates="incident", lazy="select")

class IncidentLogModel(Base):
    __tablename__ = "incident_logs"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    incident_id = Column(Integer, ForeignKey("incidents.id"), nullable=False)
    notes = Column(Text, nullable=False)
    created_at = Column(DateTime, default=datetime.utcnow, nullable=False)

    incident = relationship("IncidentModel", back_populates="logs")

Base.metadata.create_all(bind=engine)

# ------------------------------------------------------------------------------
# 2. Database Dependency
# ------------------------------------------------------------------------------
def get_db():
    db_session_basede26 = SessionLocal()
    try:
        yield db_session_basede26
    finally:
        db_session_basede26.close()

# ------------------------------------------------------------------------------
# 3. Pydantic Schemas with Format Validation
# ------------------------------------------------------------------------------
# Auth Schemas
class UserCreate(BaseModel):
    name: str
    email: EmailStr
    password: str

class UserLogin(BaseModel):
    email: EmailStr
    password: str

# Congestion Index Schemas
class CongestionIndexCreate(BaseModel):
    level_name: str = Field(..., min_length=1, max_length=100)
    description: Optional[str] = None
    index_code: str = Field(..., pattern=r"^CONG-[A-Z0-9]+-[0-9]+$")
```

```python
class CongestionIndexUpdate(BaseModel):
    level_name: Optional[str] = Field(None, min_length=1, max_length=100)
    description: Optional[str] = None
    index_code: Optional[str] = Field(None, pattern=r"^CONG-[A-Z0-9]+-[0-9]+$")

class CongestionIndexResponse(BaseModel):
    id: int
    level_name: str
    description: Optional[str] = None
    index_code: str
    created_at: datetime
    updated_at: datetime

    class Config:
        from_attributes = True

# Incident Schemas
class IncidentCreate(BaseModel):
    title: str = Field(..., min_length=1, max_length=255)
    incident_code: str = Field(..., pattern=r"^INC-\d{4}-\d{3,}$")
    delay_minutes: int = Field(0, ge=0)
    congestion_index_id: int

class IncidentUpdate(BaseModel):
    title: Optional[str] = Field(None, min_length=1, max_length=255)
    incident_code: Optional[str] = Field(None, pattern=r"^INC-\d{4}-\d{3,}$")
    delay_minutes: Optional[int] = Field(None, ge=0)
    congestion_index_id: Optional[int] = None

class IncidentResponse(BaseModel):
    id: int
    title: str
    incident_code: str
    delay_minutes: int
    congestion_index_id: int
    created_at: datetime
    updated_at: datetime

    class Config:
        from_attributes = True

# Log Schemas
class LogResponse(BaseModel):
    id: int
    notes: str
    class Config:
        from_attributes = True

class IncidentWithLogsResponse(BaseModel):
    id: int
    title: str
```

```python
    id: int
    title: str
    incident_code: str
    logs: List[LogResponse]
    class Config:
        from_attributes = True

# ------------------------------------------------------------------------------
# 4. FastAPI Setup
# ------------------------------------------------------------------------------
app = FastAPI(title="Municipal Transit Incident Manager")
app.mount("/static", StaticFiles(directory="static"), name="static")

# ------------------------------------------------------------------------------
# 5. Session Authentication Helper
# ------------------------------------------------------------------------------
def get_current_user(request: Request, db_session_basede26: Session = Depends(get_db)) -> UserModel:
    session_token = request.cookies.get("session_id")
    if not session_token:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Not authenticated")

    session_record = db_session_basede26.query(SessionModel).filter(
        SessionModel.id == session_token
    ).first()

    if not session_record:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid session")

    if datetime.utcnow() > session_record.expires_at:
        db_session_basede26.delete(session_record)
        db_session_basede26.commit()
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Session expired")

    return session_record.user

# ------------------------------------------------------------------------------
# 6. Auth Endpoints
# ------------------------------------------------------------------------------
@app.post("/register", status_code=status.HTTP_201_CREATED)
def register(user_data: UserCreate, db_session_basede26: Session = Depends(get_db)):
    existing_user = db_session_basede26.query(UserModel).filter(UserModel.email == user_data.email).first()
    if existing_user:
        raise HTTPException(status_code=400, detail="Email already registered")

    salt = bcrypt.gensalt()
    hashed_pw = bcrypt.hashpw(user_data.password.encode('utf-8'), salt).decode('utf-8')

    new_user = UserModel(name=user_data.name, email=user_data.email, password_hash=hashed_pw)
    db_session_basede26.add(new_user)
    db_session_basede26.commit()
    return {"message": "User registered successfully"}

```

```python
@app.post("/login")
def login(login_data: UserLogin, response: Response, db_session_basede26: Session = Depends(get_db)):
    user = db_session_basede26.query(UserModel).filter(UserModel.email == login_data.email).first()
    if not user or not bcrypt.checkpw(login_data.password.encode('utf-8'), user.password_hash.encode('utf-8')):
        raise HTTPException(status_code=401, detail="Invalid email or password")

    session_token = str(uuid.uuid4())
    created_at = datetime.utcnow()
    expires_at = created_at + timedelta(hours=8)

    session_record = SessionModel(
        id=session_token, user_id=user.id, created_at=created_at, expires_at=expires_at
    )
    db_session_basede26.add(session_record)
    db_session_basede26.commit()

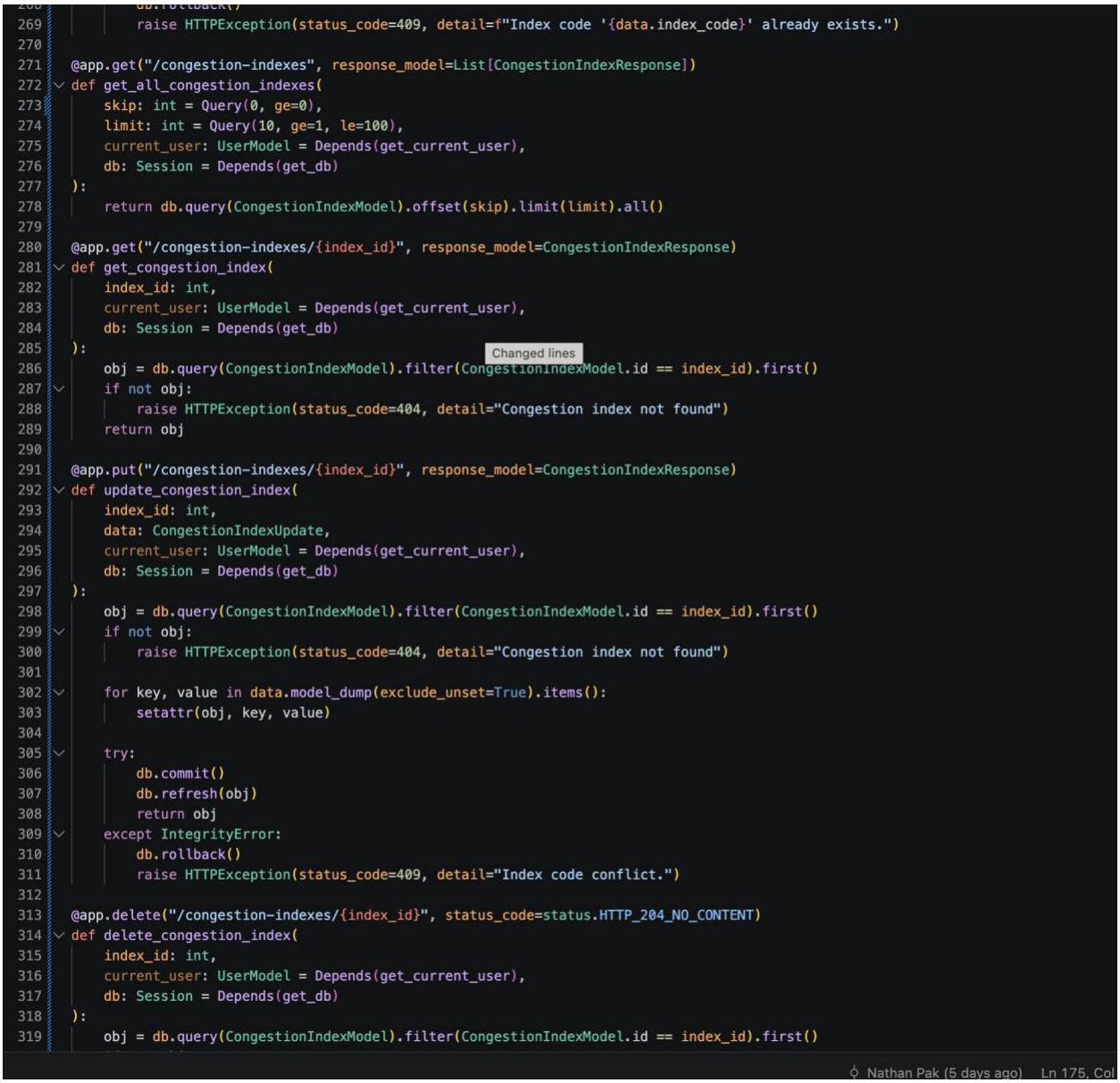
    response.set_cookie(key="session_id", value=session_token, httponly=True, samesite="lax", secure=False)
    return {"email": user.email, "name": user.name}

@app.post("/logout")
def logout(request: Request, response: Response, db_session_basede26: Session = Depends(get_db)):
    session_token = request.cookies.get("session_id")
    if session_token:
        db_session_basede26.query(SessionModel).filter(SessionModel.id == session_token).delete()
        db_session_basede26.commit()

    response.delete_cookie("session_id")
    return {"message": "Logged out successfully"}

@app.get("/me")
def get_me(current_user: UserModel = Depends(get_current_user)):
    return {"email": current_user.email, "name": current_user.name}

# ---------------------------------------------------------------------------
# 7. Secondary Entity CRUD Endpoints (Congestion Indexes)
# ---------------------------------------------------------------------------
@app.post("/congestion-indexes", response_model=CongestionIndexResponse, status_code=status.HTTP_201_CREATED)
def create_congestion_index(
    data: CongestionIndexCreate,
    current_user: UserModel = Depends(get_current_user),
    db: Session = Depends(get_db)
):
    try:
        index_obj = CongestionIndexModel(**data.model_dump())
        db.add(index_obj)
        db.commit()
        db.refresh(index_obj)
        return index_obj
    except IntegrityError:
        db.rollback()
```


raise HTTPException(status_code=409, detail=f"Index code '{data.index_code}' already exists.")
@app.get("/congestion-indexes", response_model=List[CongestionIndexResponse])
def get_all_congestion_indexes(
skip: int = Query(0, ge=0),
limit: int = Query(10, ge=1, le=100),
current_user: UserModel = Depends(get_current_user),
db: Session = Depends(get_db)
):
return db.query(CongestionIndexModel).offset(skip).limit(limit).all()
@app.get("/congestion-indexes/{index_id}", response_model=CongestionIndexResponse)
def get_congestion_index(
index_id: int,
current_user: UserModel = Depends(get_current_user),
db: Session = Depends(get_db)
):
Changed lines
obj = db.query(CongestionIndexModel).filter(CongestionIndexModel.id == index_id).first()
if not obj:
raise HTTPException(status_code=404, detail="Congestion index not found")
return obj
@app.put("/congestion-indexes/{index_id}", response_model=CongestionIndexResponse)
def update_congestion_index(
index_id: int,
data: CongestionIndexUpdate,
current_user: UserModel = Depends(get_current_user),
db: Session = Depends(get_db)
):
obj = db.query(CongestionIndexModel).filter(CongestionIndexModel.id == index_id).first()
if not obj:
raise HTTPException(status_code=404, detail="Congestion index not found")
for key, value in data.model_dump(exclude_unset=True).items():
setattr(obj, key, value)
try:
db.commit()
db.refresh(obj)
return obj
except IntegrityError:
db.rollback()
raise HTTPException(status_code=409, detail="Index code conflict.")
@app.delete("/congestion-indexes/{index_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_congestion_index(
index_id: int,
current_user: UserModel = Depends(get_current_user),
db: Session = Depends(get_db)
):
obj = db.query(CongestionIndexModel).filter(CongestionIndexModel.id == index_id).first()
Nathan Pak (5 days ago)
Ln 175, Col

```python
        )
    obj = db.query(CongestionIndexModel).filter(CongestionIndexModel.id == index_id).first()
    if not obj:
        raise HTTPException(status_code=404, detail="Congestion index not found")

    try:
        db.delete(obj)
        db.commit()
    except IntegrityError:
        db.rollback()
        raise HTTPException(
            status_code=400,
            detail="Cannot delete congestion index with existing associated transit incidents."
        )

# ---------------------------------------------------------------------------
# 8. Primary Entity CRUD Endpoints (Transit Incidents)
# ---------------------------------------------------------------------------
@app.post("/incidents", response_model=IncidentResponse, status_code=status.HTTP_201_CREATED)
def create_incident(
    data: IncidentCreate,
    current_user: UserModel = Depends(get_current_user),
    db: Session = Depends(get_db)
):
    related = db.query(CongestionIndexModel).filter(CongestionIndexModel.id == data.congestion_index_id).first()
    if not related:
        raise HTTPException(status_code=400, detail=f"Congestion index ID {data.congestion_index_id} does not exist.")

    try:
        incident = IncidentModel(**data.model_dump())
        db.add(incident)
        db.commit()
        db.refresh(incident)
        return incident
    except IntegrityError:
        db.rollback()
        raise HTTPException(status_code=409, detail=f"Incident code '{data.incident_code}' already exists.")

@app.get("/incidents", response_model=List[IncidentResponse])
def get_all_incidents(
    skip: int = Query(0, ge=0),
    limit: int = Query(10, ge=1, le=100),
    current_user: UserModel = Depends(get_current_user),
    db: Session = Depends(get_db)
):
    return db.query(IncidentModel).offset(skip).limit(limit).all()

@app.get("/incidents/{incident_id}", response_model=IncidentResponse)
def get_incident_by_id(
    incident_id: int,
    current_user: UserModel = Depends(get_current_user),
    db: Session = Depends(get_db)
):
```

```python
    incident = db.query(IncidentModel).filter(IncidentModel.id == incident_id).first()
    if not incident:
        raise HTTPException(status_code=404, detail="Incident record not found")
    return incident

@app.put("/incidents/{incident_id}", response_model=IncidentResponse)
def update_incident(
    incident_id: int,
    data: IncidentUpdate,
    current_user: UserModel = Depends(get_current_user),
    db: Session = Depends(get_db)
):
    incident = db.query(IncidentModel).filter(IncidentModel.id == incident_id).first()
    if not incident:
        raise HTTPException(status_code=404, detail="Incident record not found")

    if data.congestion_index_id is not None:
        related = db.query(CongestionIndexModel).filter(CongestionIndexModel.id == data.congestion_index_id).first()
        if not related:
            raise HTTPException(status_code=400, detail="Target congestion index ID does not exist.")

    for key, value in data.model_dump(exclude_unset=True).items():
        setattr(incident, key, value)

    try:
        db.commit()
        db.refresh(incident)
        return incident
    except IntegrityError:
        db.rollback()
        raise HTTPException(status_code=409, detail="Incident code conflict.")

@app.delete("/incidents/{incident_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_incident(
    incident_id: int,
    current_user: UserModel = Depends(get_current_user),
    db: Session = Depends(get_db)
):
    incident = db.query(IncidentModel).filter(IncidentModel.id == incident_id).first()
    if not incident:
        raise HTTPException(status_code=404, detail="Incident record not found")

    db.delete(incident)
    db.commit()

# ---------------------------------------------------------------------------
# 9. Relationship Query Endpoint
# ---------------------------------------------------------------------------
@app.get("/congestion-indexes/{index_id}/incidents", response_model=List[IncidentResponse])
def get_incidents_by_congestion_index(
    index_id: int,
```

```python
def get_incidents_by_congestion_index(
    index_id: int,
    current_user: UserModel = Depends(get_current_user),
    db: Session = Depends(get_db)
):
    parent = db.query(CongestionIndexModel).filter(CongestionIndexModel.id == index_id).first()
    if not parent:
        raise HTTPException(status_code=404, detail="Congestion index not found")

    return db.query(IncidentModel).filter(IncidentModel.congestion_index_id == index_id).all()

# ------------------------------------------------------------------------------
# 10. Frontend Catch-All Route
# ------------------------------------------------------------------------------
@app.get("/{full_path:path}")
def serve_react_app(full_path: str):
    return FileResponse("templates/index.html")

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("main:app", host="0.0.0.0", port=8166, reload=True)
```

Schemas.py

schemas.py > ...

```python
from pydantic import BaseModel, Field
from datetime import datetime
from typing import Optional, List

# --- Congestion Index Schemas ---
class CongestionIndexBase(BaseModel):
    level_name: str = Field(..., min_length=1, max_length=100)
    description: Optional[str] = None
    # Format pattern validation: e.g., CONG-HIGH-01
    index_code: str = Field(
        ...,
        pattern=r"^CONG-[A-Z0-9]+-[0-9]+$",
        description="Format: CONG-LEVEL-NUMBER (e.g., CONG-HIGH-01)"
    )

class CongestionIndexCreate(CongestionIndexBase):
    pass

class CongestionIndexUpdate(BaseModel):
    level_name: Optional[str] = Field(None, min_length=1, max_length=100)
    description: Optional[str] = None
    index_code: Optional[str] = Field(None, pattern=r"^CONG-[A-Z0-9]+-[0-9]+$")

class CongestionIndexResponse(CongestionIndexBase):
    id: int
    created_at: datetime
    updated_at: datetime

    class Config:
        from_attributes = True


# --- Transit Incident Schemas ---
class TransitIncidentBase(BaseModel):
    title: str = Field(..., min_length=1, max_length=255)
    # Format pattern validation: e.g., INC-2026-001
    incident_code: str = Field(
        ...,
        pattern=r"^INC-\d{4}-\d{3,}$",
        description="Format: INC-YYYY-NUMBER (e.g., INC-2026-001)"
    )
    delay_minutes: int = Field(0, ge=0)
    congestion_index_id: int

class TransitIncidentCreate(TransitIncidentBase):
    pass

class TransitIncidentUpdate(BaseModel):
    title: Optional[str] = Field(None, min_length=1, max_length=255)
    incident_code: Optional[str] = Field(None, pattern=r"^INC-\d{4}-\d{3,}$")
    delay_minutes: Optional[int] = Field(None, ge=0)
    congestion_index_id: Optional[int] = None

```

```python

class TransitIncidentResponse(TransitIncidentBase):
    id: int
    created_at: datetime
    updated_at: datetime

    class Config:
        from_attributes = True
```

Database.py

```python
database.py > get_db
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

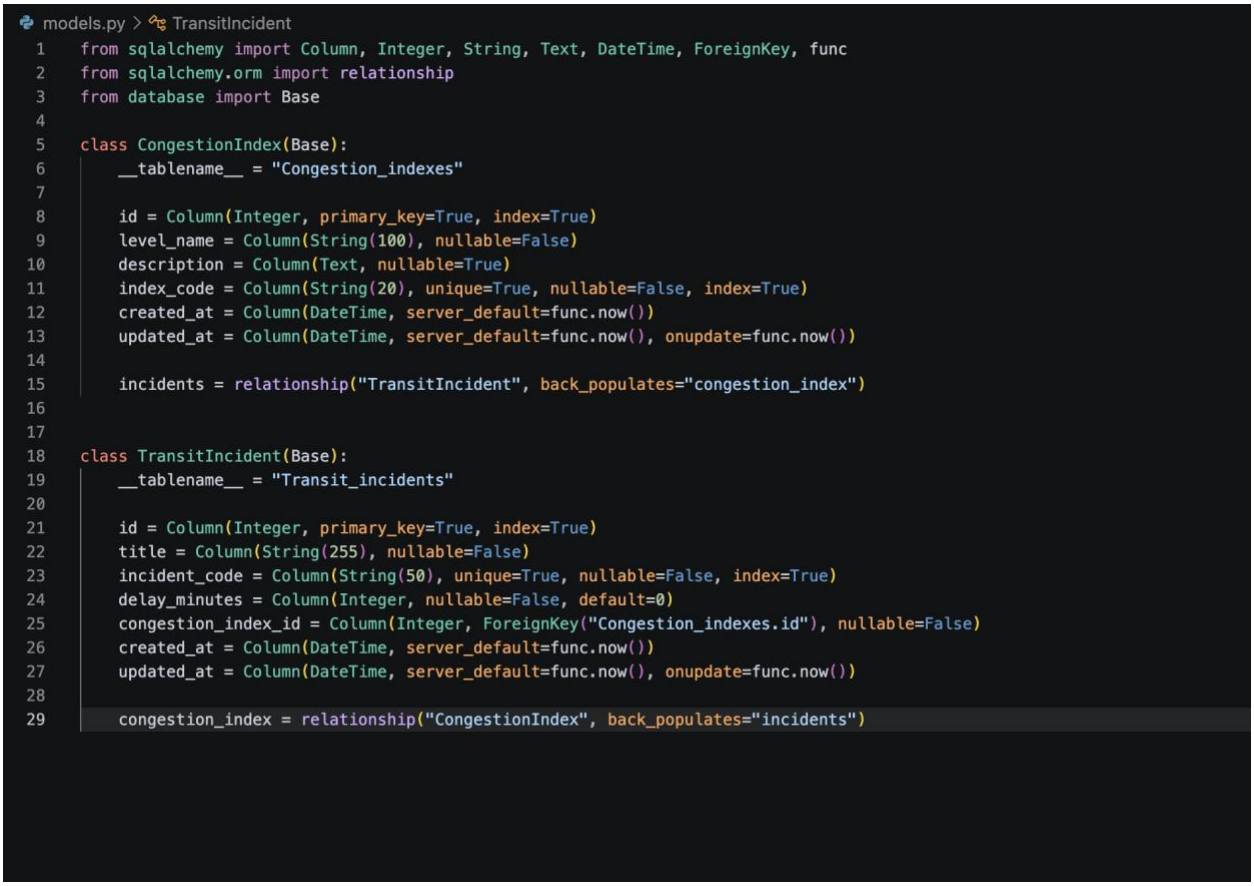
SQLALCHEMY_DATABASE_URL = "mysql+pymysql://root:password@localhost:3306/Municipal_rel"

engine = create_engine(SQLALCHEMY_DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

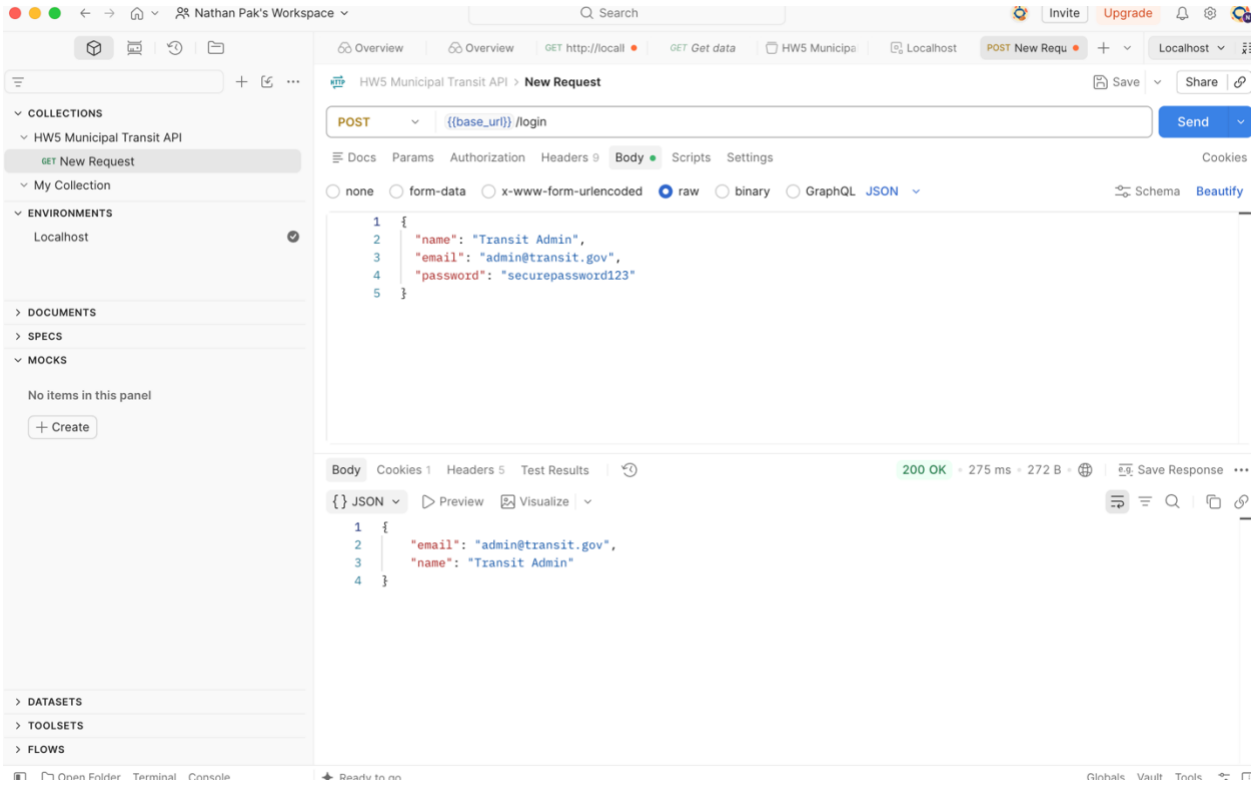
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

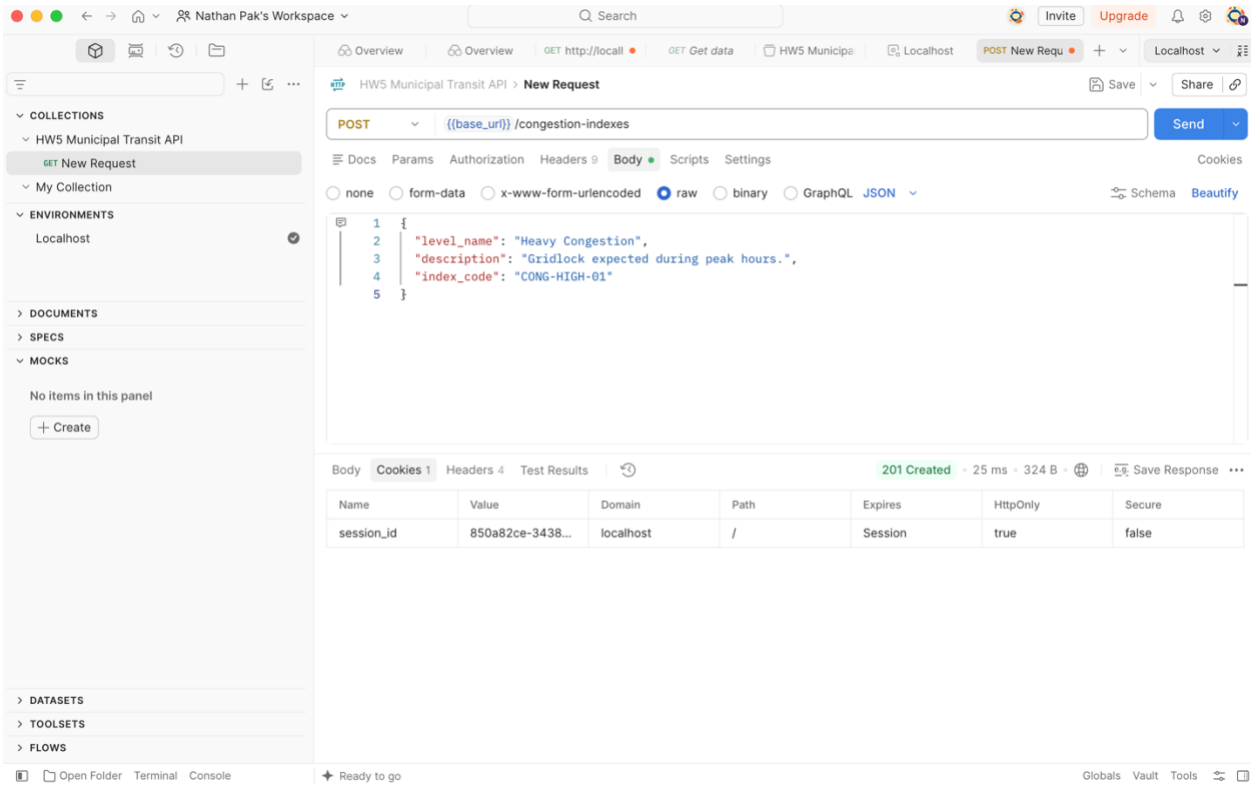
Models.py


models.py > TransitIncident
from sqlalchemy import Column, Integer, String, Text, DateTime, ForeignKey, func
from sqlalchemy.orm import relationship
from database import Base
class CongestionIndex(Base):
__tablename__ = "Congestion_indexes"
id = Column(Integer, primary_key=True, index=True)
level_name = Column(String(100), nullable=False)
description = Column(Text, nullable=True)
index_code = Column(String(20), unique=True, nullable=False, index=True)
created_at = Column(DateTime, server_default=func.now())
updated_at = Column(DateTime, server_default=func.now(), onupdate=func.now())
incidents = relationship("TransitIncident", back_populates="congestion_index")
class TransitIncident(Base):
__tablename__ = "Transit_incidents"
id = Column(Integer, primary_key=True, index=True)
title = Column(String(255), nullable=False)
incident_code = Column(String(50), unique=True, nullable=False, index=True)
delay_minutes = Column(Integer, nullable=False, default=0)
congestion_index_id = Column(Integer, ForeignKey("Congestion_indexes.id"), nullable=False)
created_at = Column(DateTime, server_default=func.now())
updated_at = Column(DateTime, server_default=func.now(), onupdate=func.now())
congestion_index = relationship("CongestionIndex", back_populates="incidents")

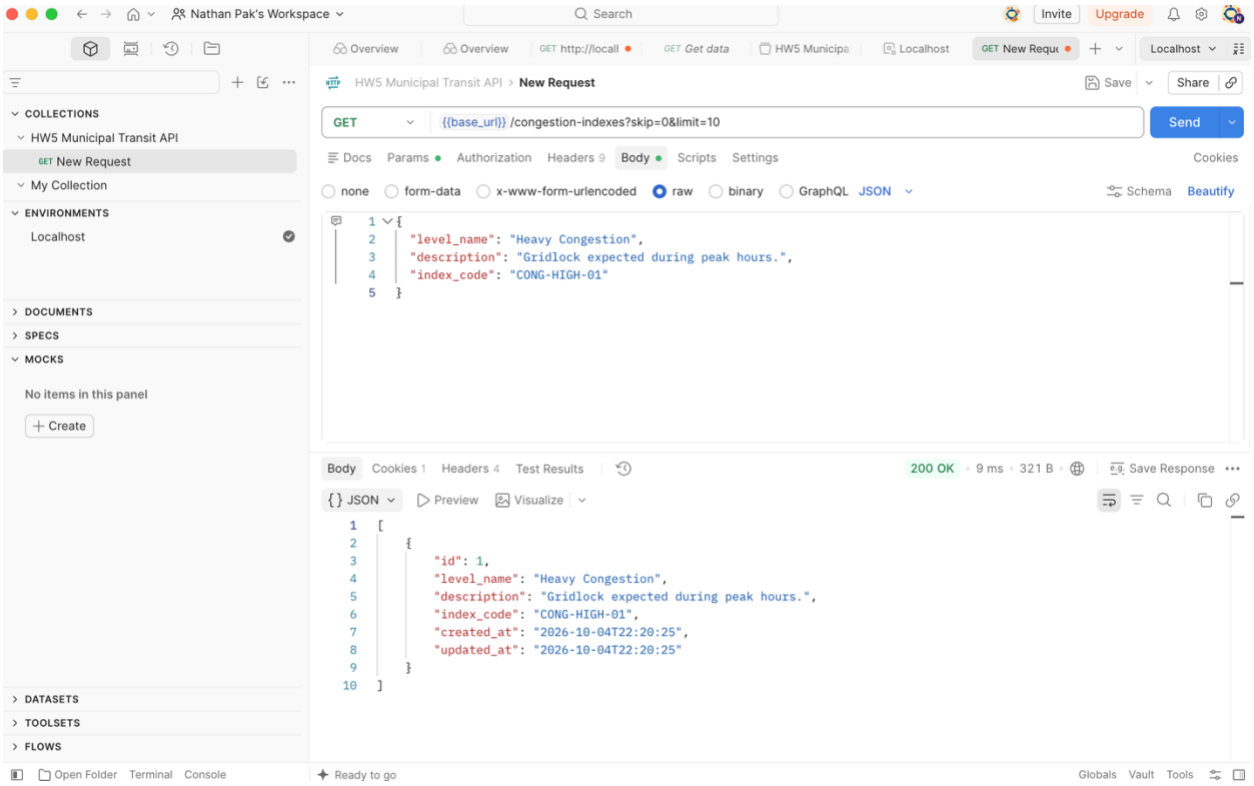
POST /login



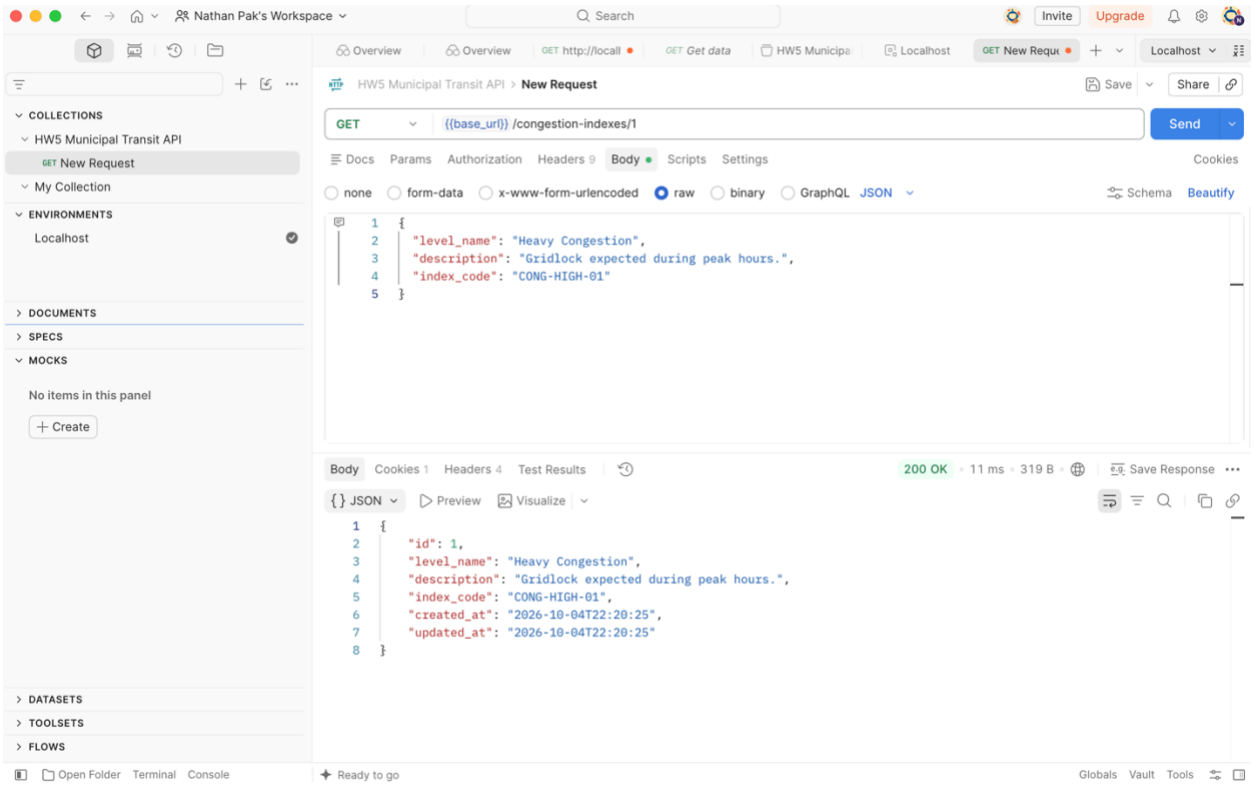
POST /congestion-indexes



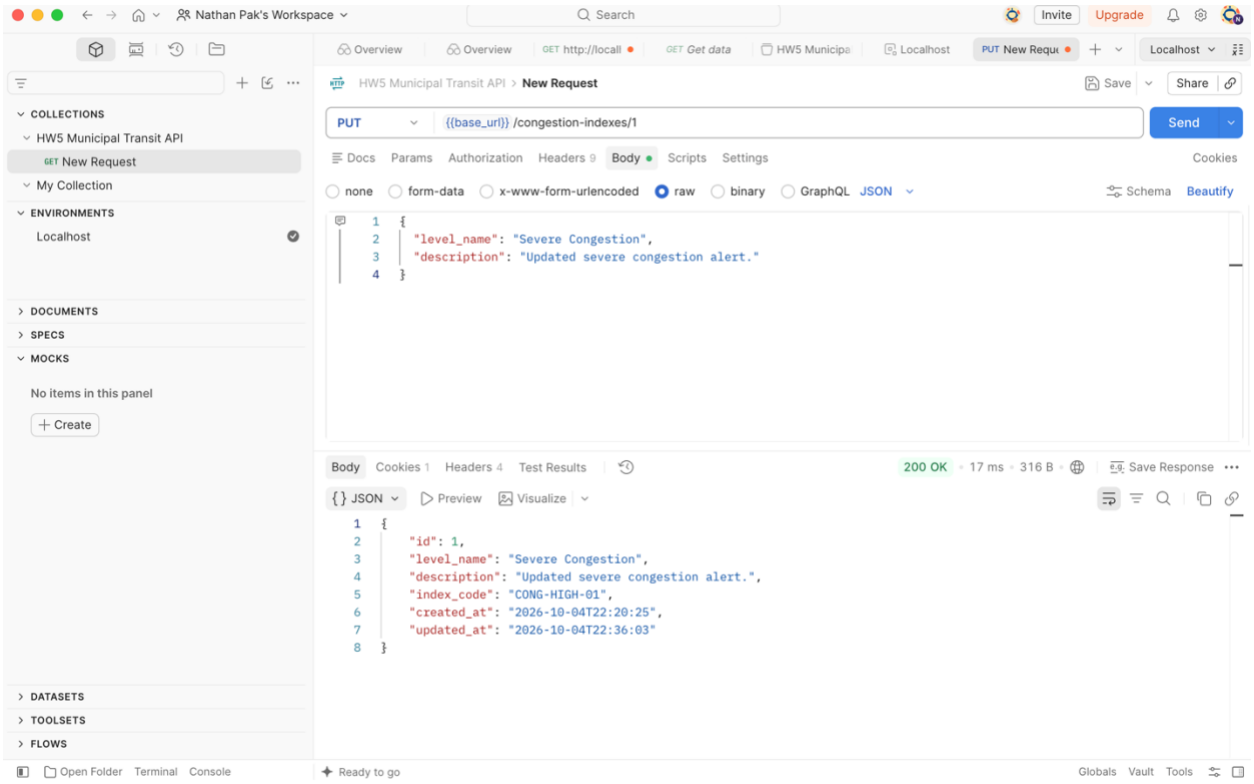
GET Pagination



GET Single Congestion Index

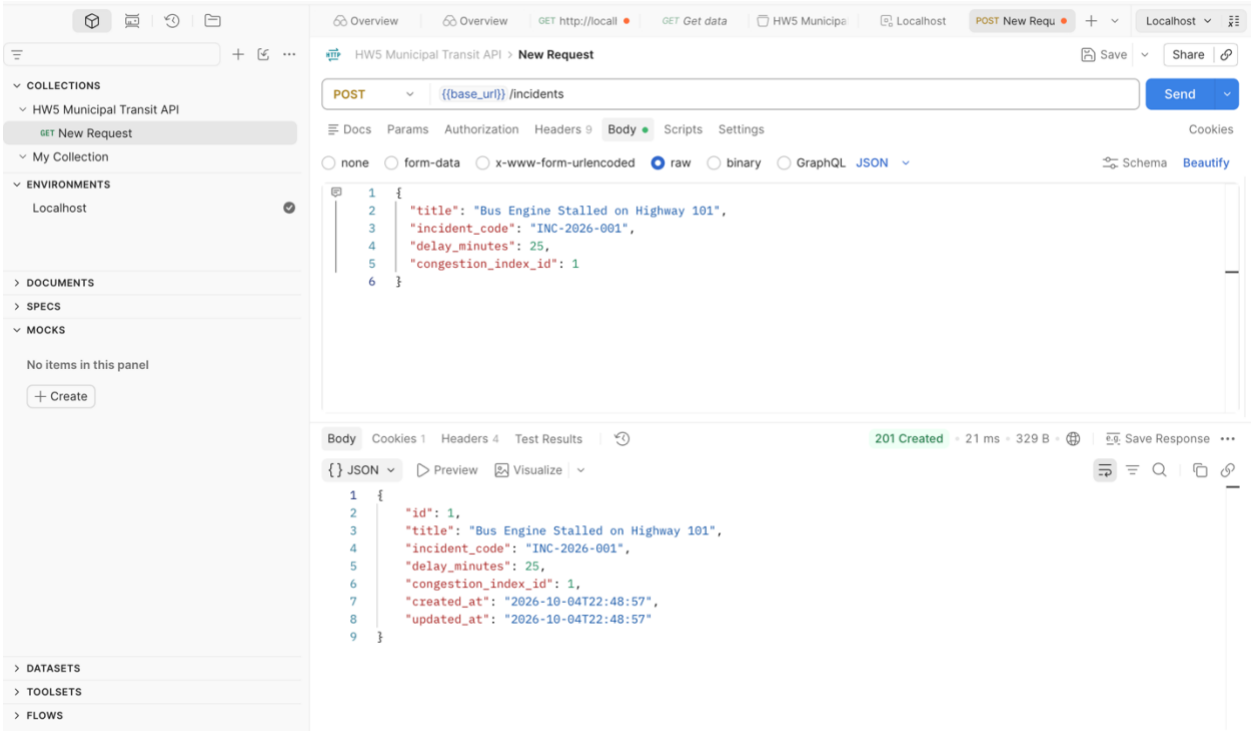


PUT Update Congestion Index

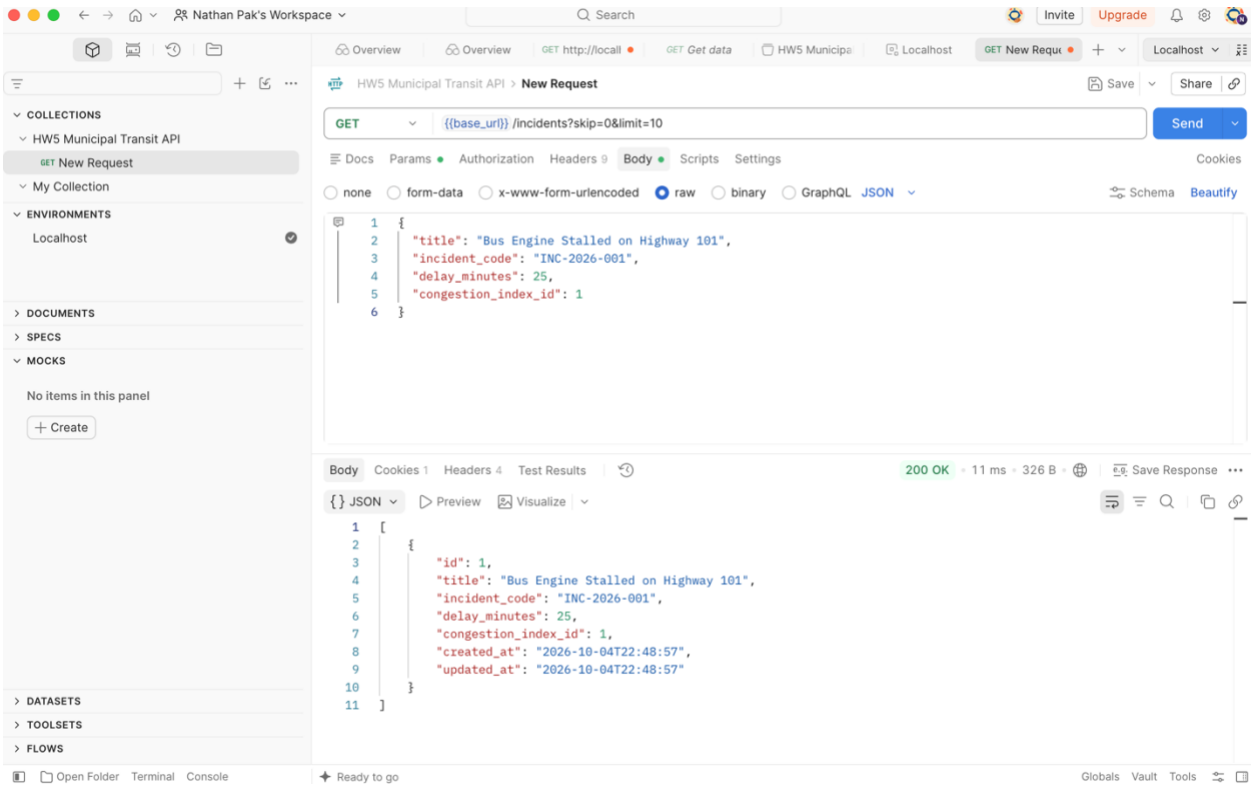


Primary Entity Requests: /incidents

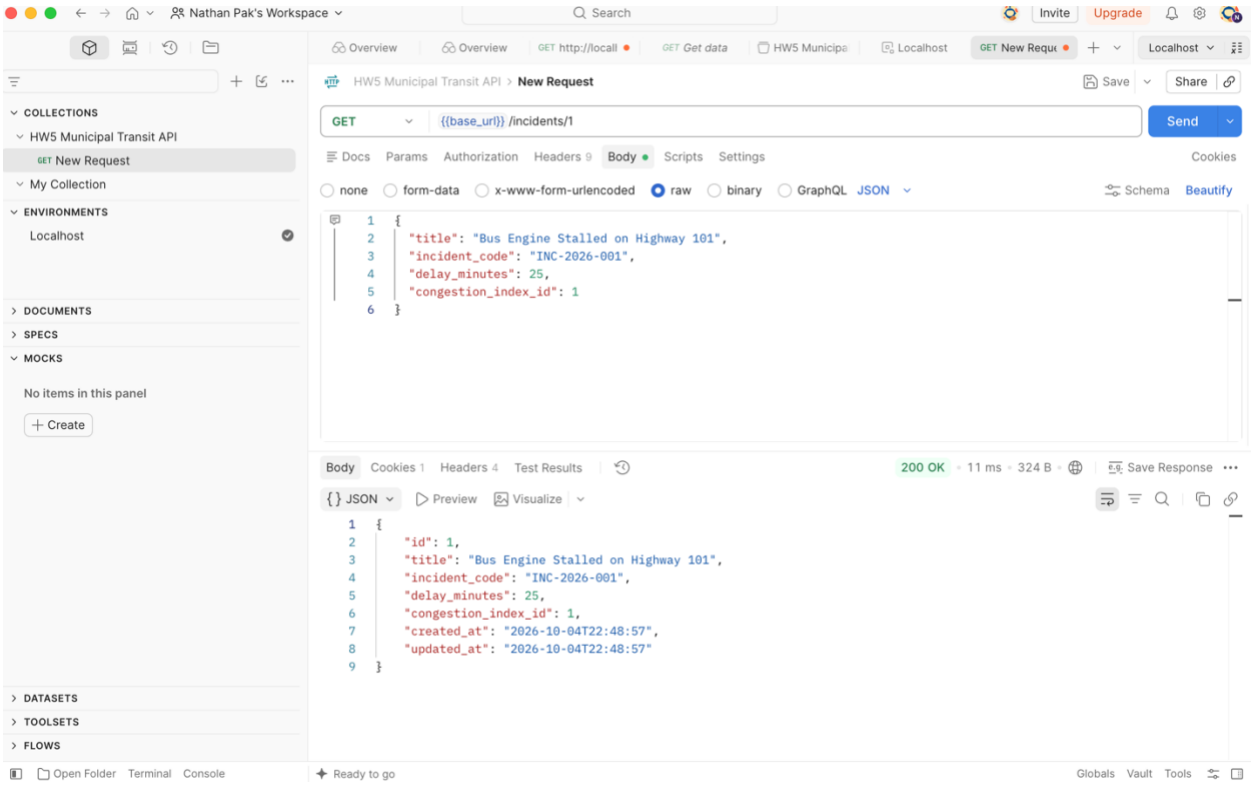
POST Create Incident



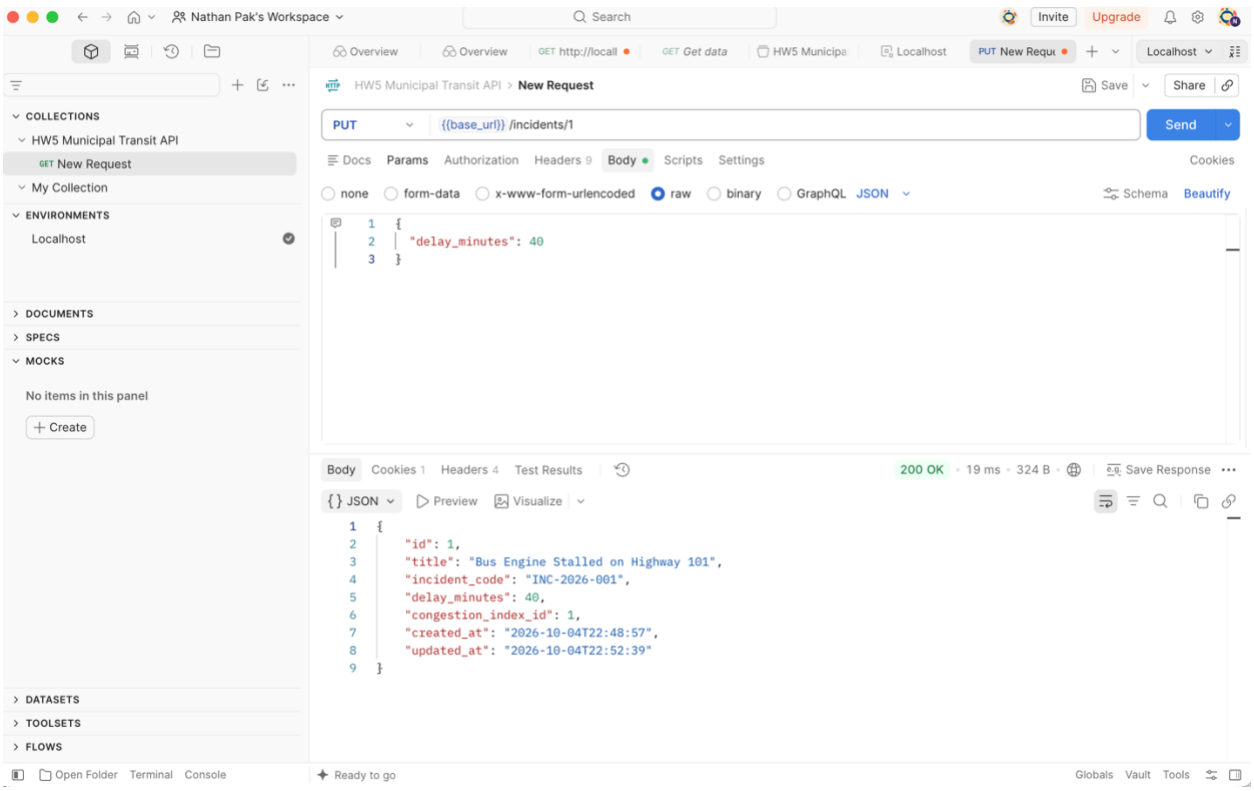
GET List incidents with Pagination



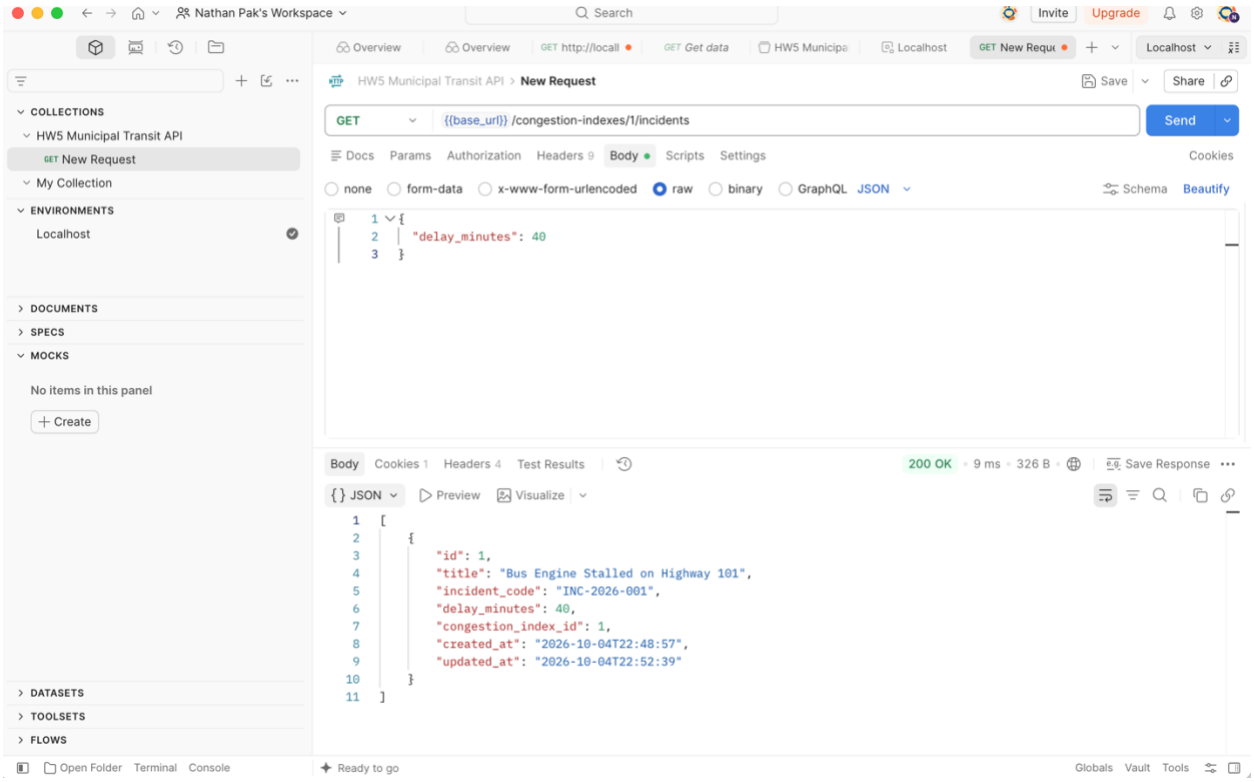
GET Single Incident



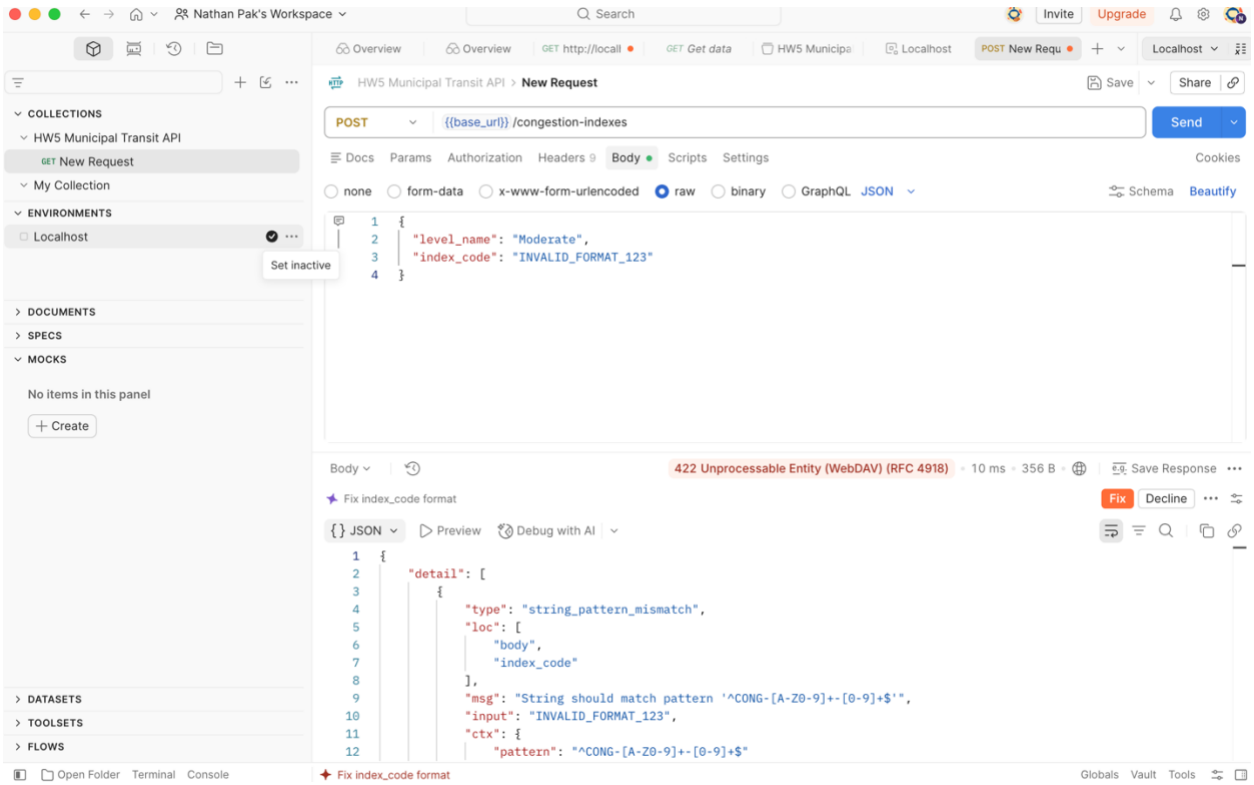
PUT Update Incident



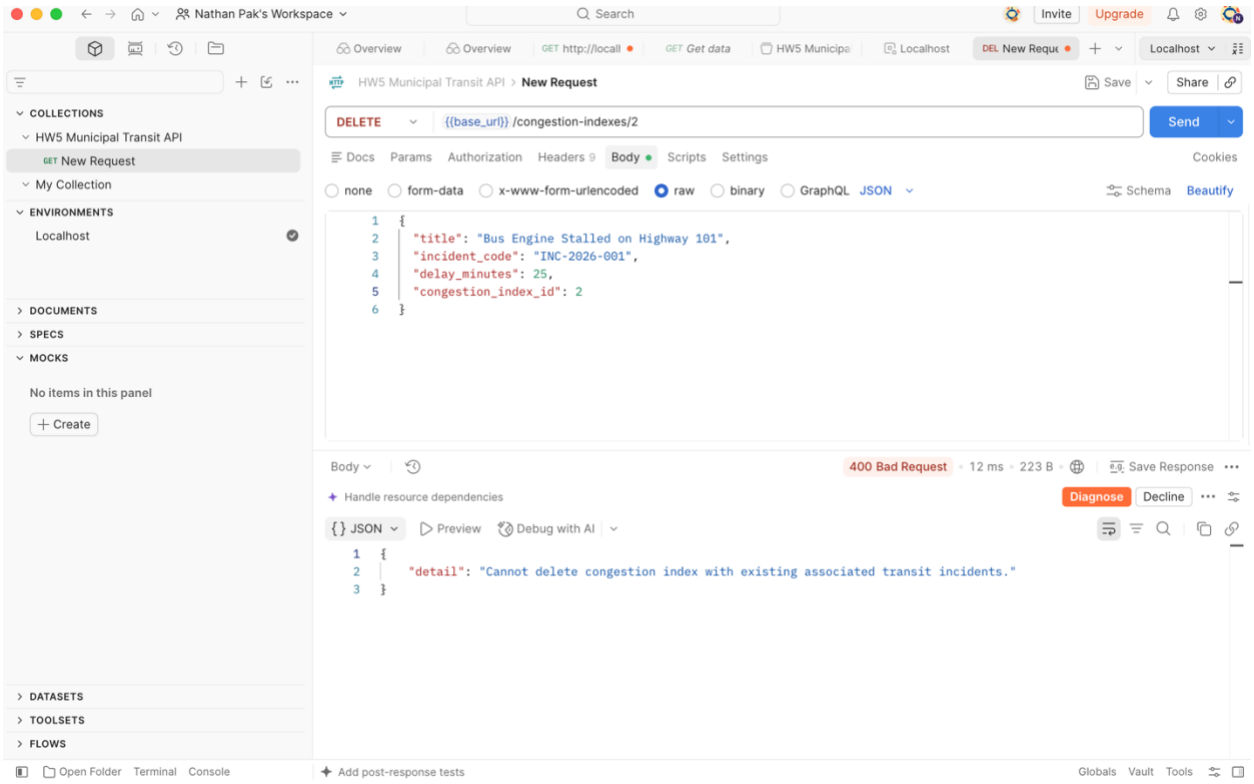
Relationship Query – Returning List containing incident INC-2026-001



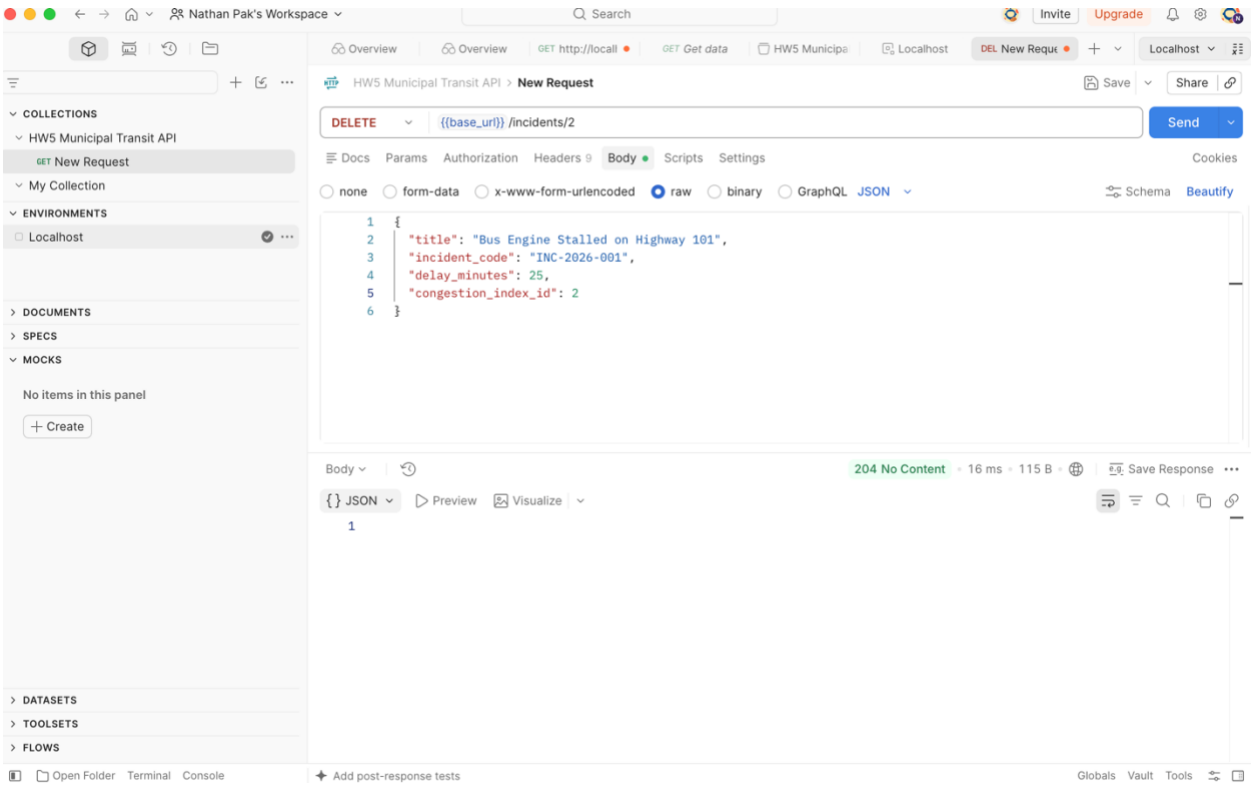
Validation Error Test – Bad Regex Format



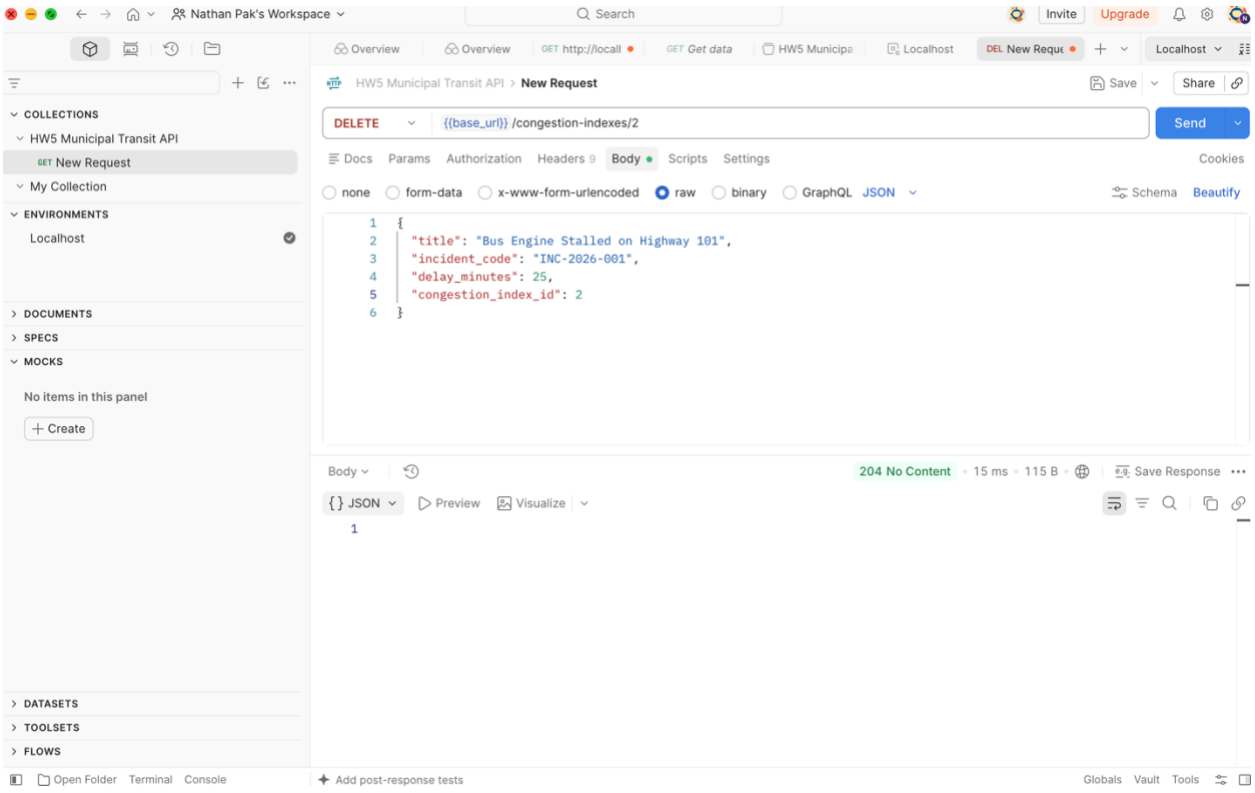
Foreign Key Deletion Constraint Test (DELETE)



Delete Incident

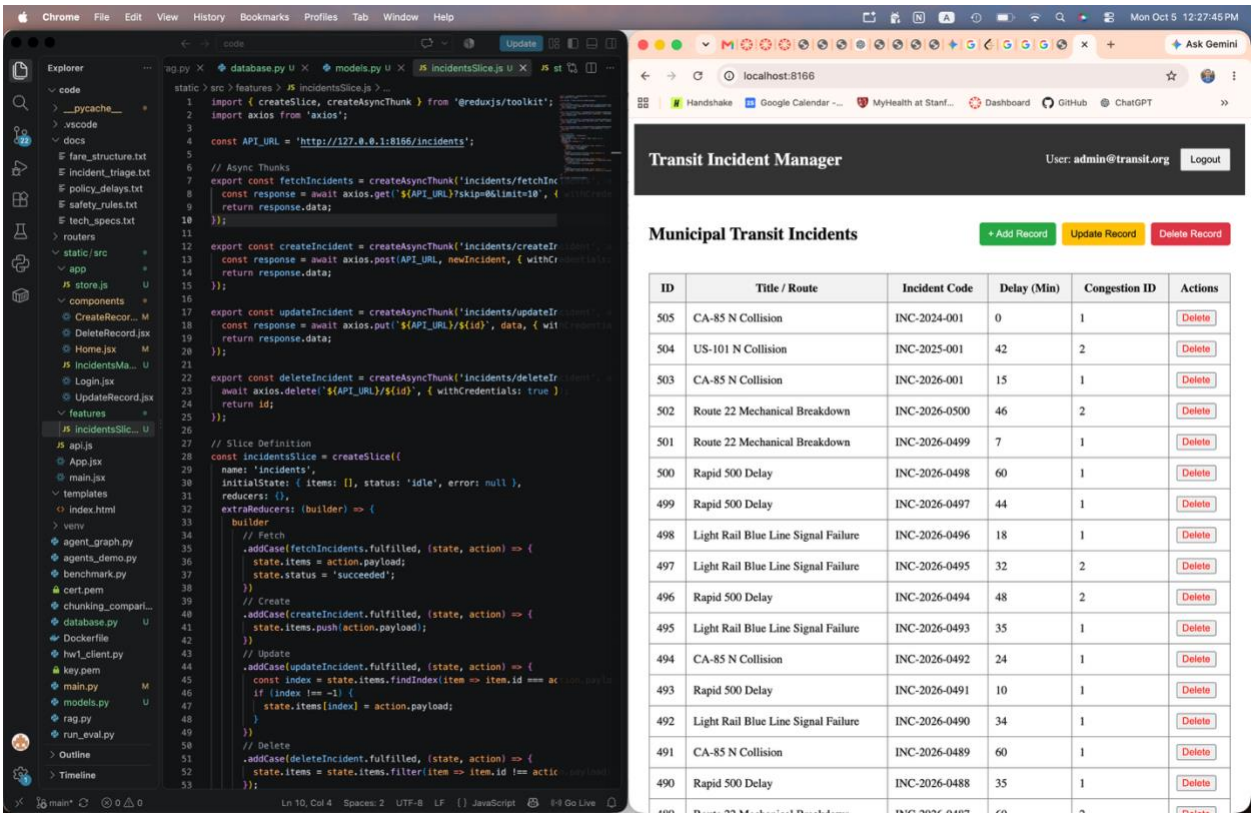


Successful Deletion of Unlinked Parent

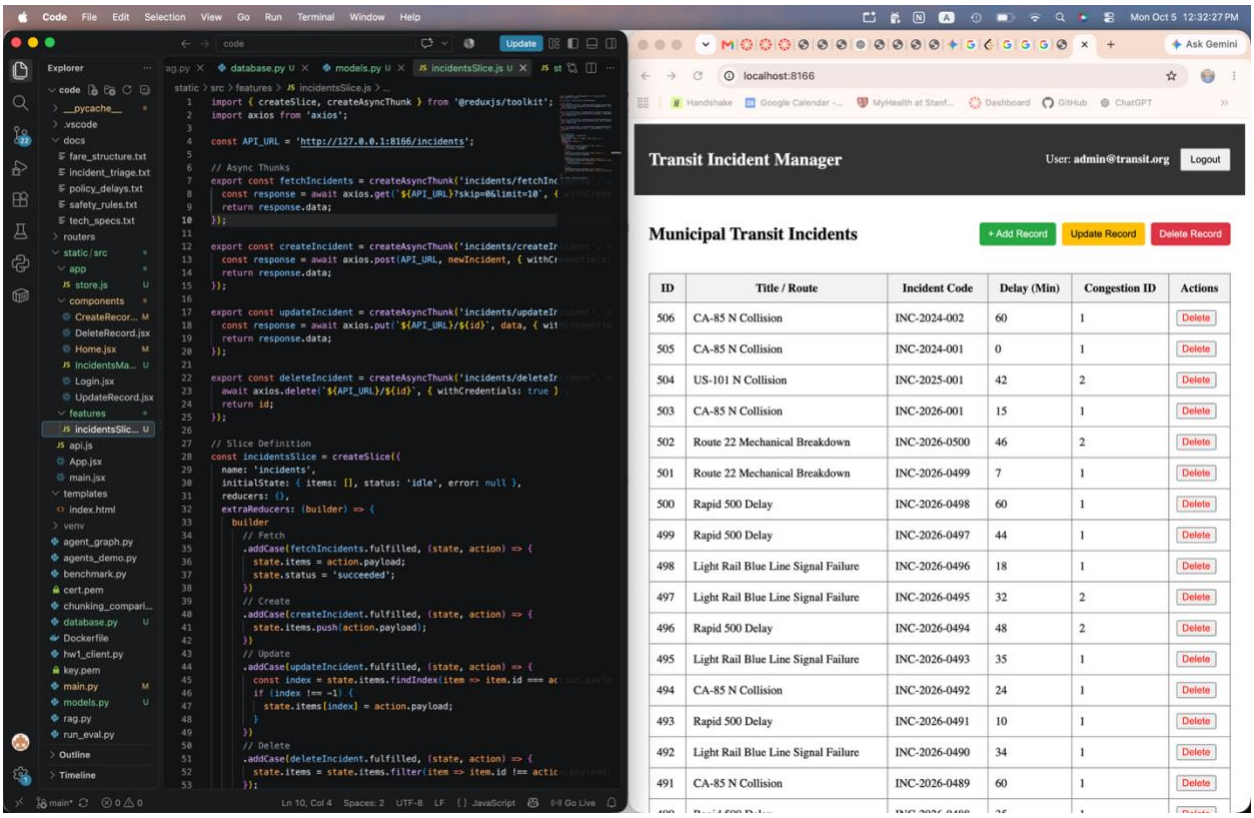


III) Redux Client

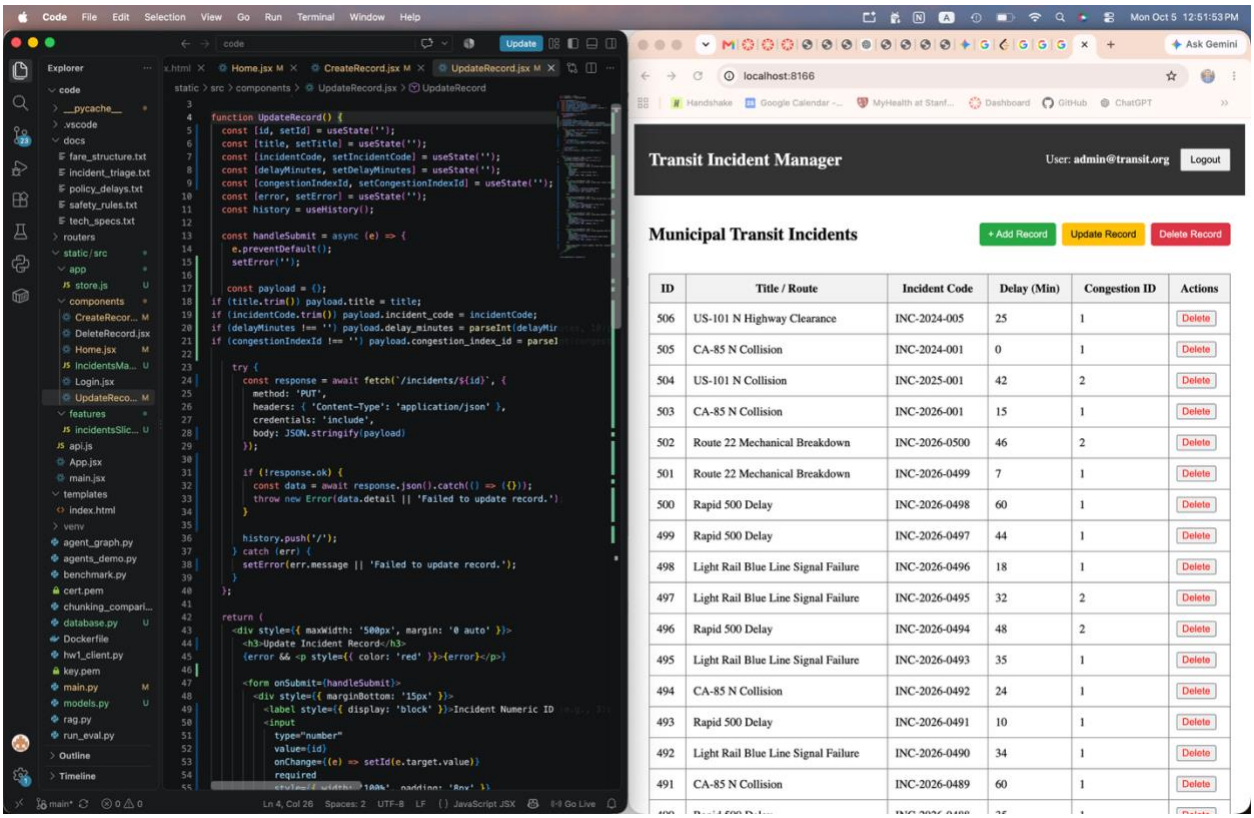
Home Screen



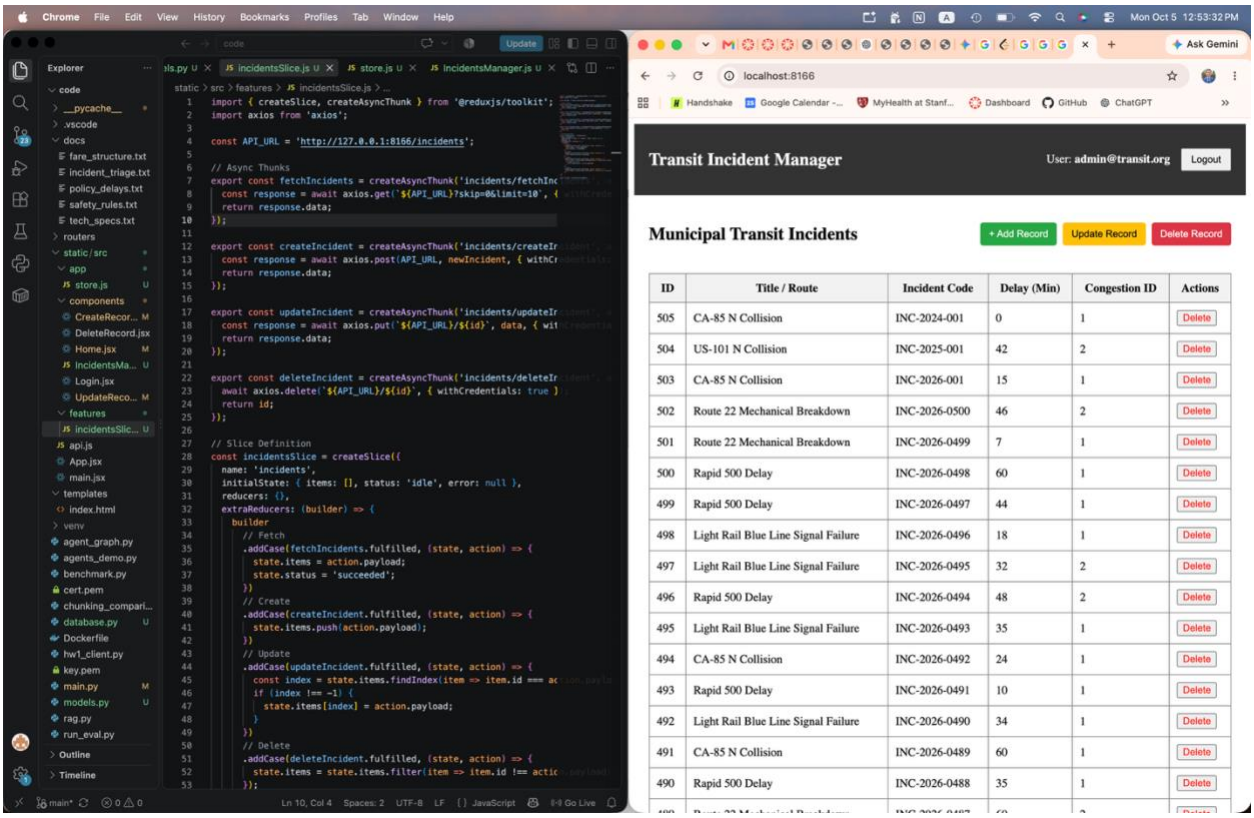
Create Screen



Updated Incident ID 506



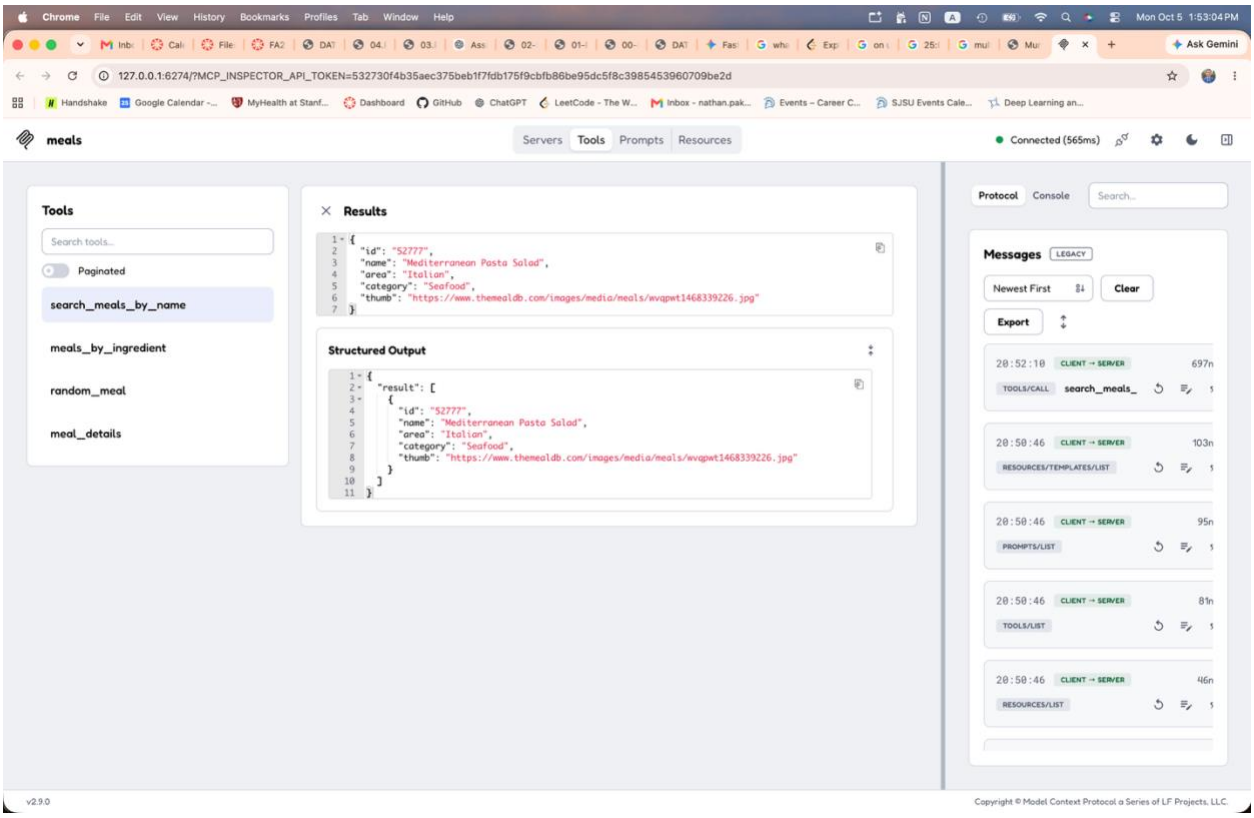
Deleted Incident ID 506



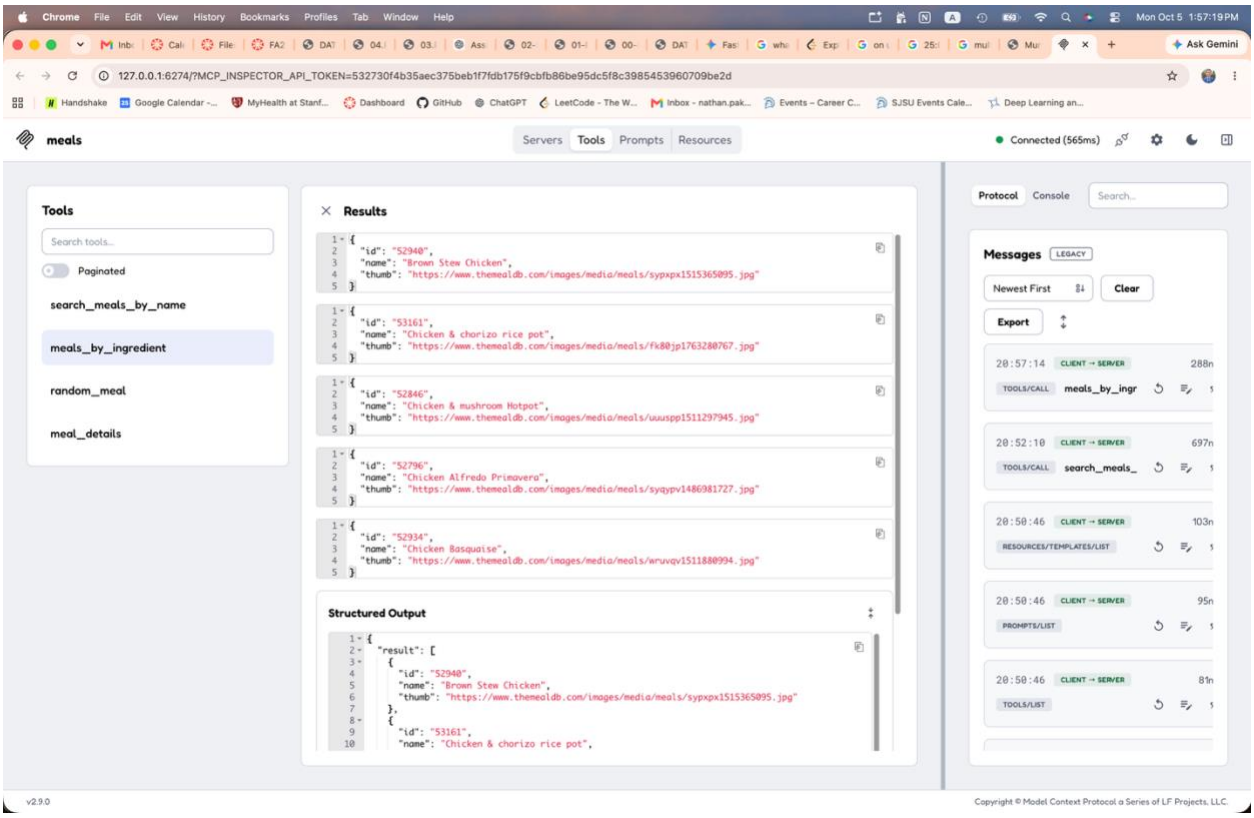
Part 2)

A)

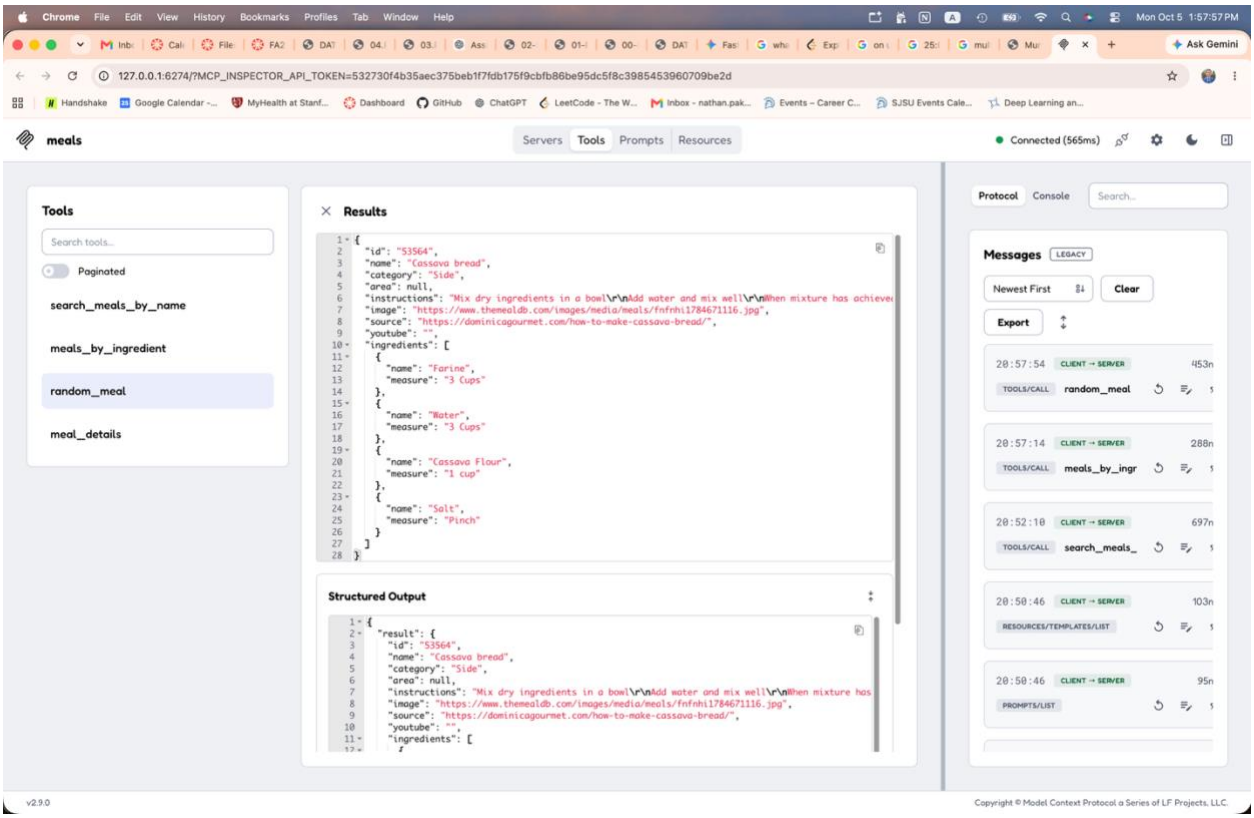
Search_meals_by_name() Query=”pasta”



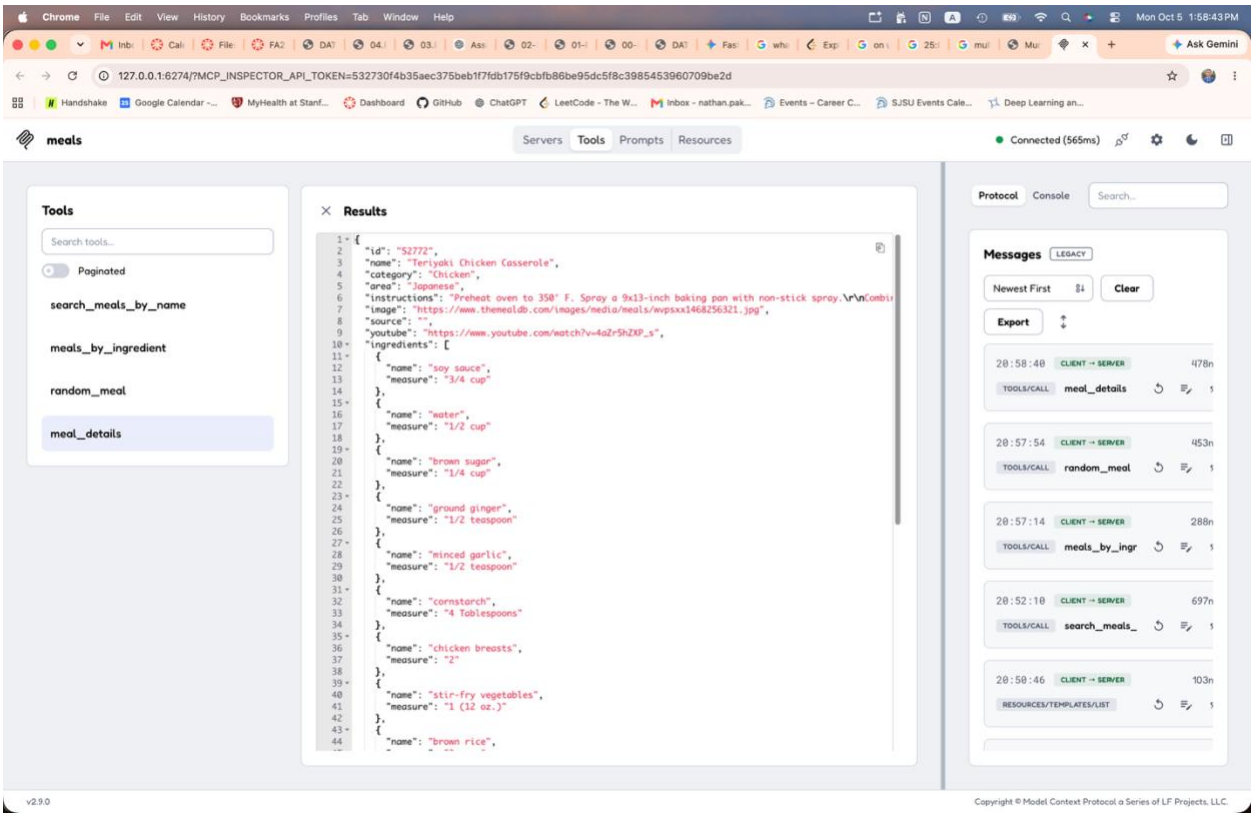
Meals_by_ingredient() – query = ‘chicken’



Random_meal()

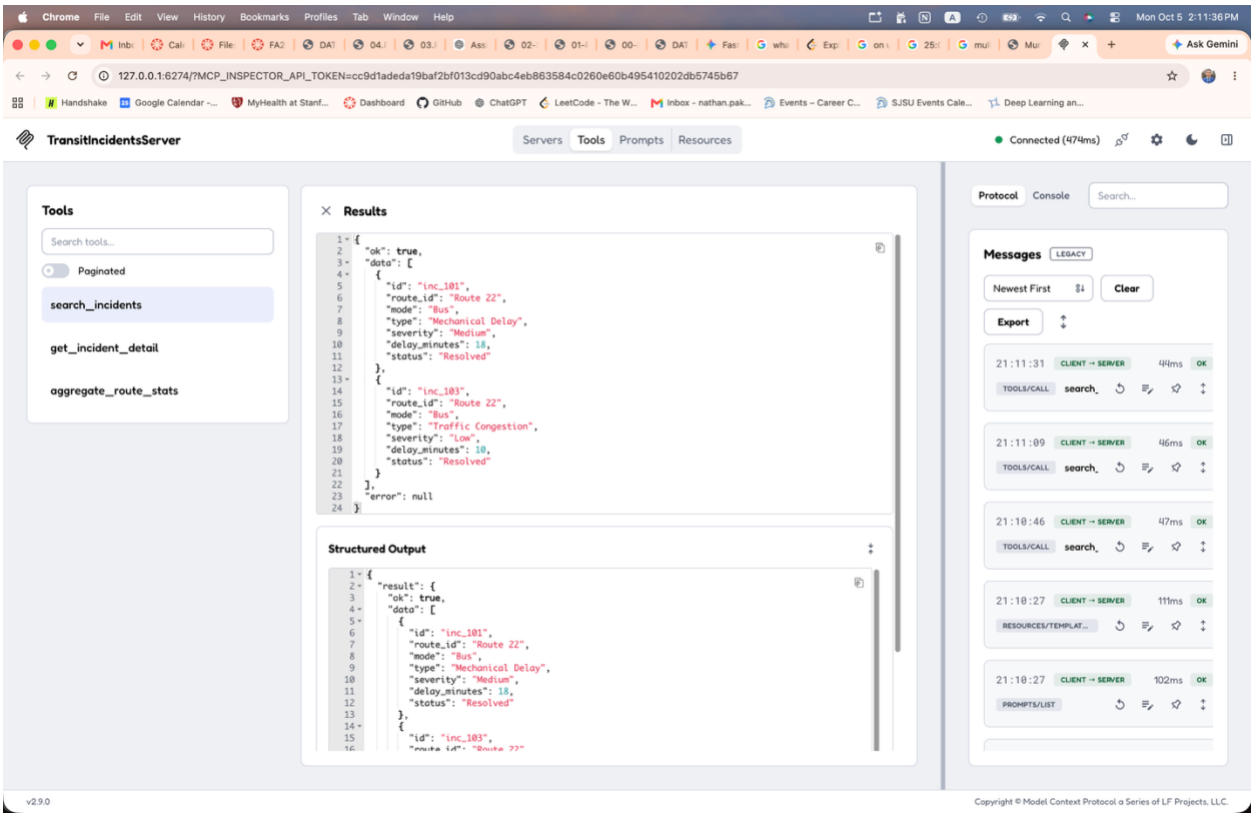


Meal_details ID: 52772

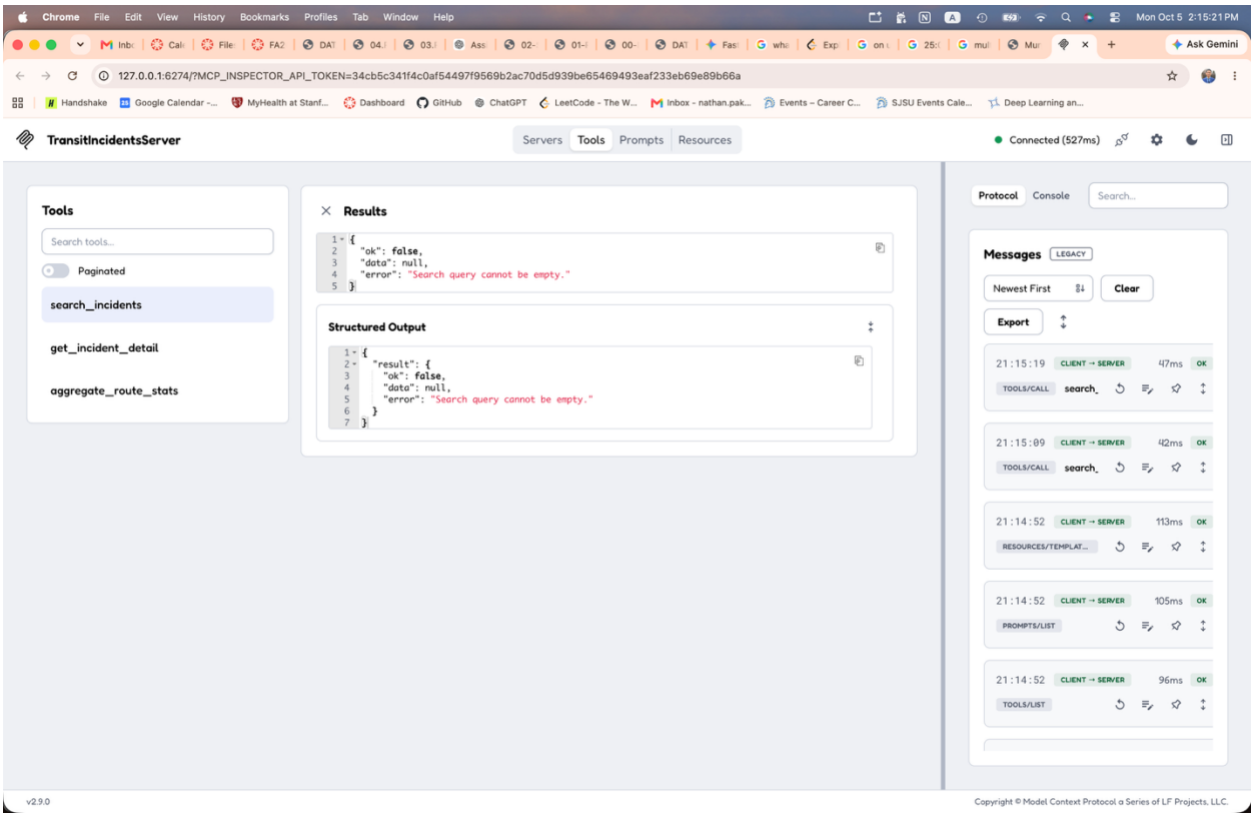


B)

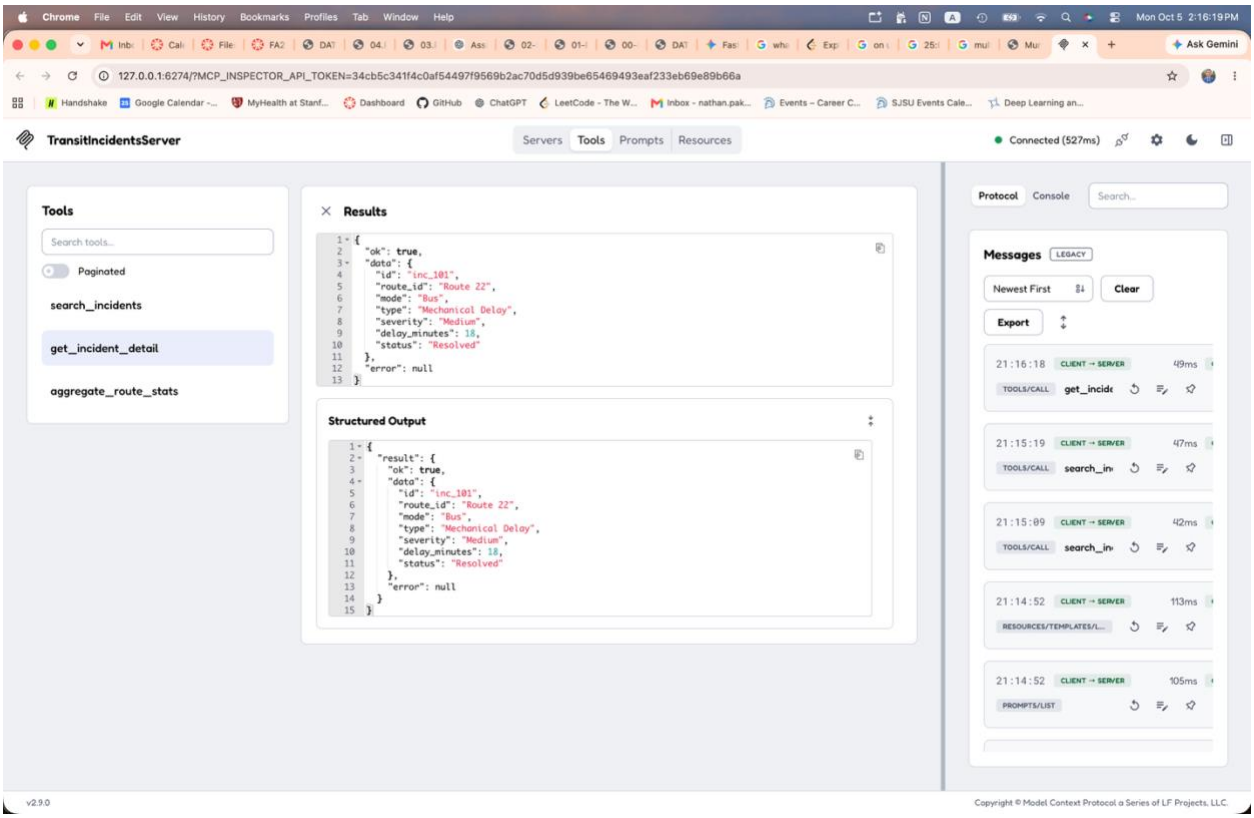
Search_items query = Route 22



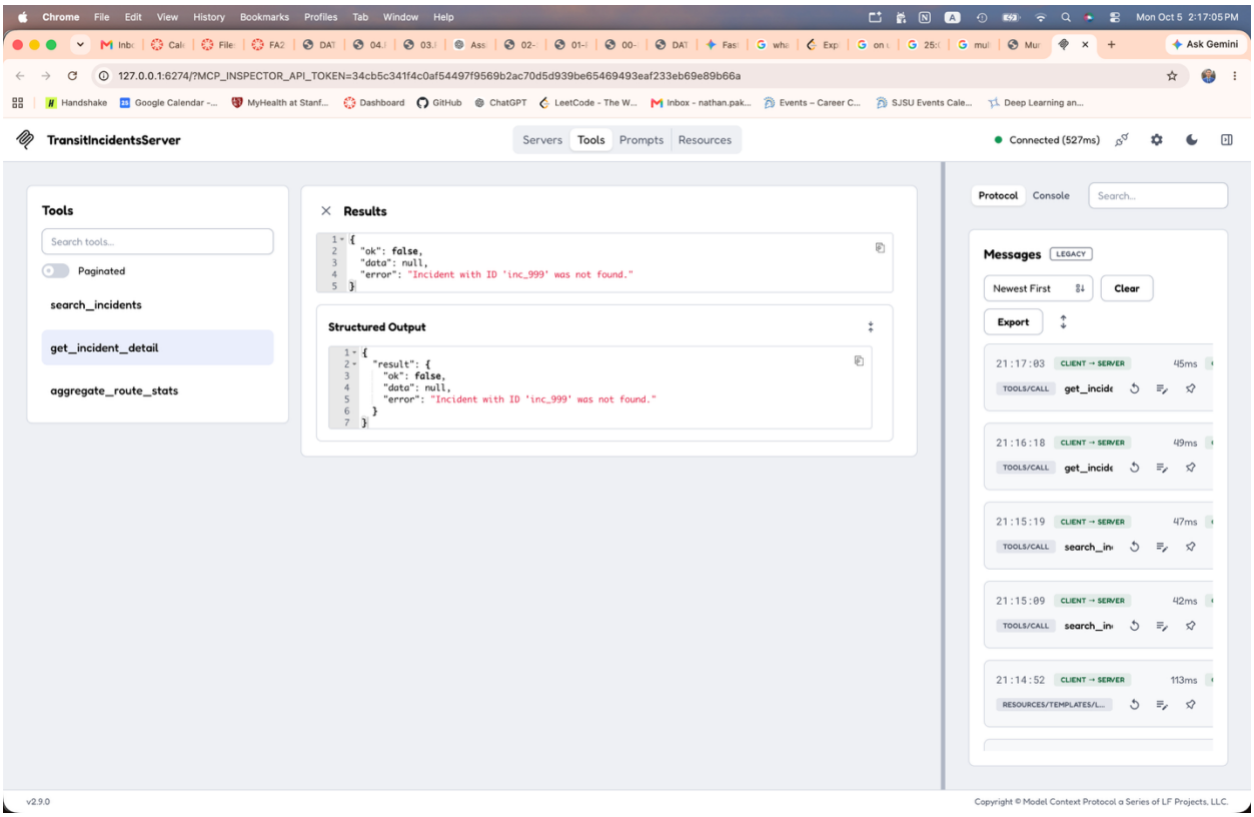
Search_items invalid input



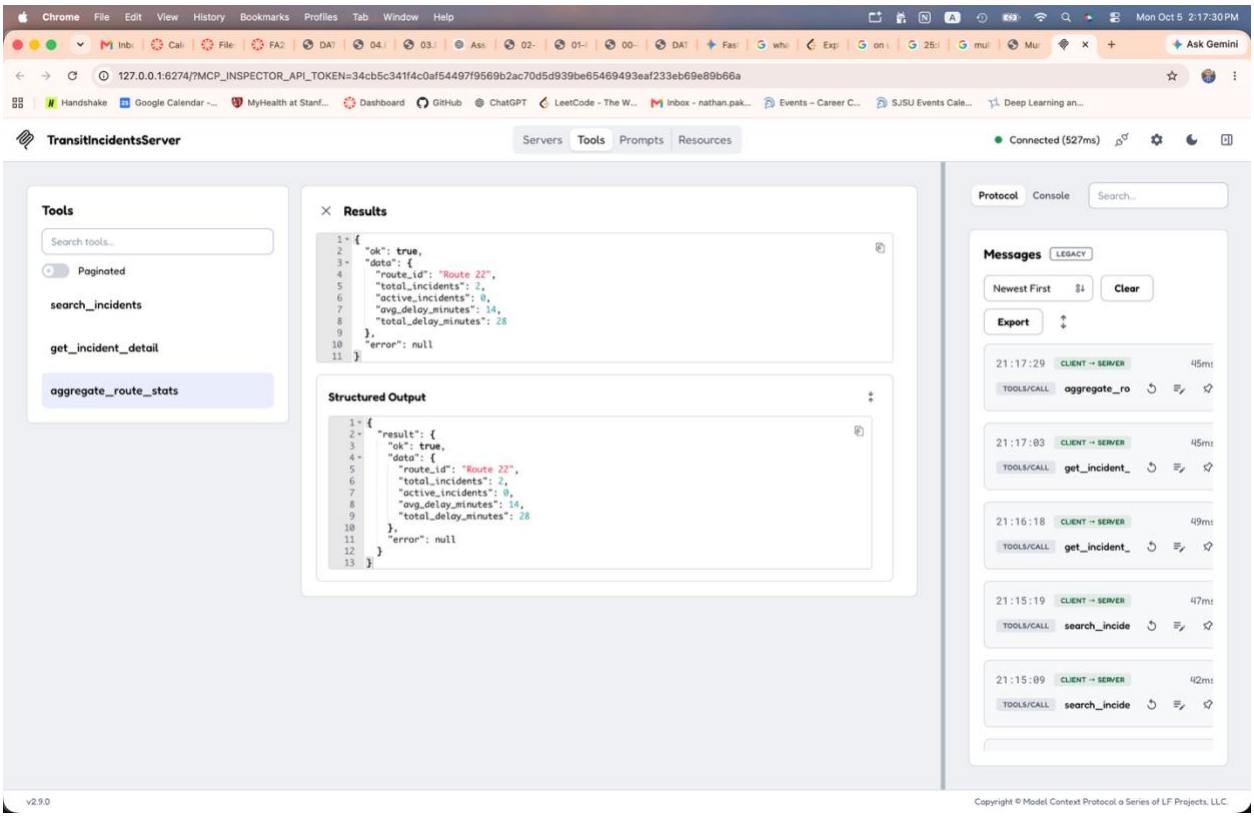
Get_incident_detail: query = inc_101



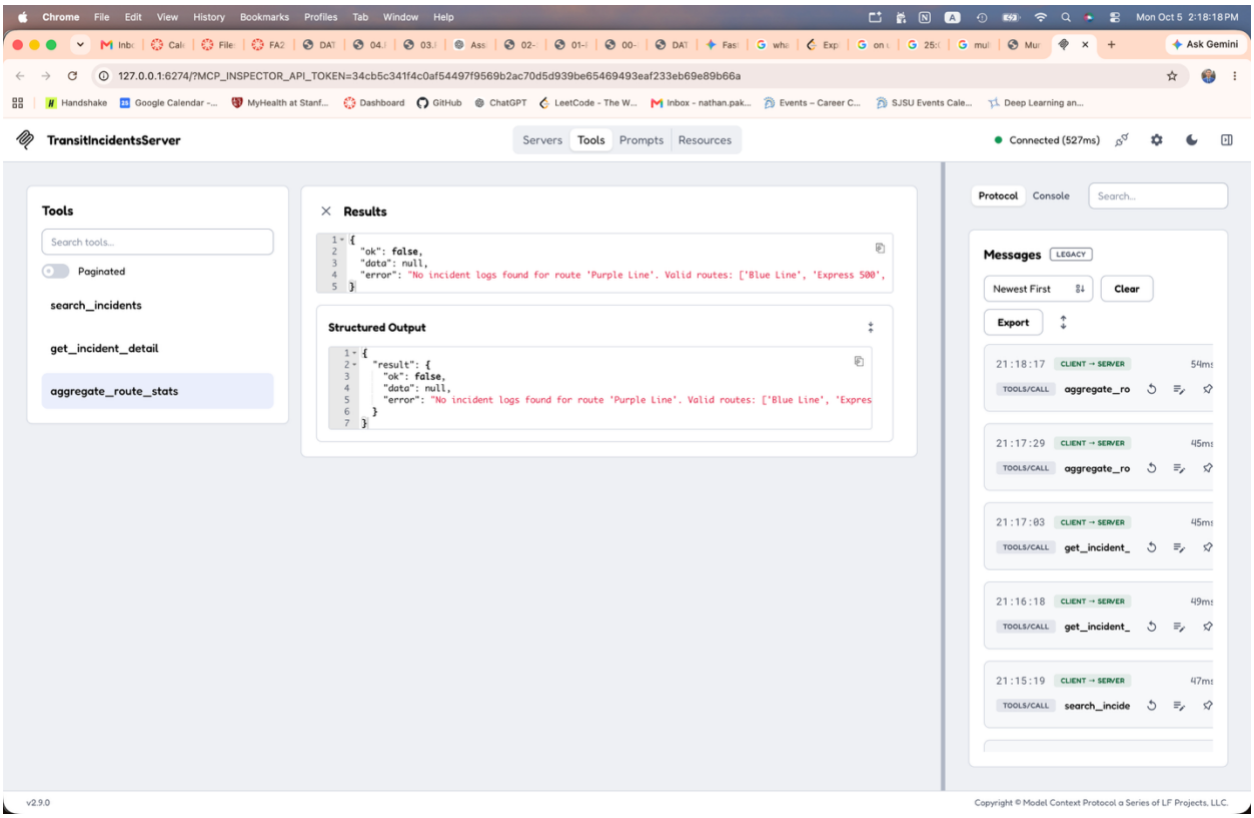
Invalid Input for get_incident_detail



Aggregate_route_stats query = Route 22



Invalid input for aggregate_route_stats



Part 3)

18)

Search_incidents

Expected JSON Input

```
{
  "type": "object",
  "properties": {
    "query": { "type": "string" }
  },
  "required": ["query"]
}
```

Rejected JSON Input

{
  "query": ""
}

Returned Error Output

{
  "ok": false,
  "data": null,
  "error": "Search query cannot be empty."
}

Reason for Rejection: The input contained an empty query string. The server enforces input validation requiring non-whitespace search terms to prevent unbounded or ambiguous scans.

Get_incident_detail

Expected Input

{
  "type": "object",
  "properties": {
    "incident_id": { "type": "string" }
  },
  "required": ["incident_id"]
}

Rejected Input

{
  "incident_id": "inc_999"
}

Returned Error

{
"ok": false,
"data": null,
"error": "Incident with ID 'inc_999' was not found."
}

The provided incident_id does not exist in the municipal transit incident dataset.

Aggregate_route_stats

Expected Input:

{
"type": "object",
"properties": {
"route_id": { "type": "string" }
},
"required": ["route_id"]
}

Rejected Input:

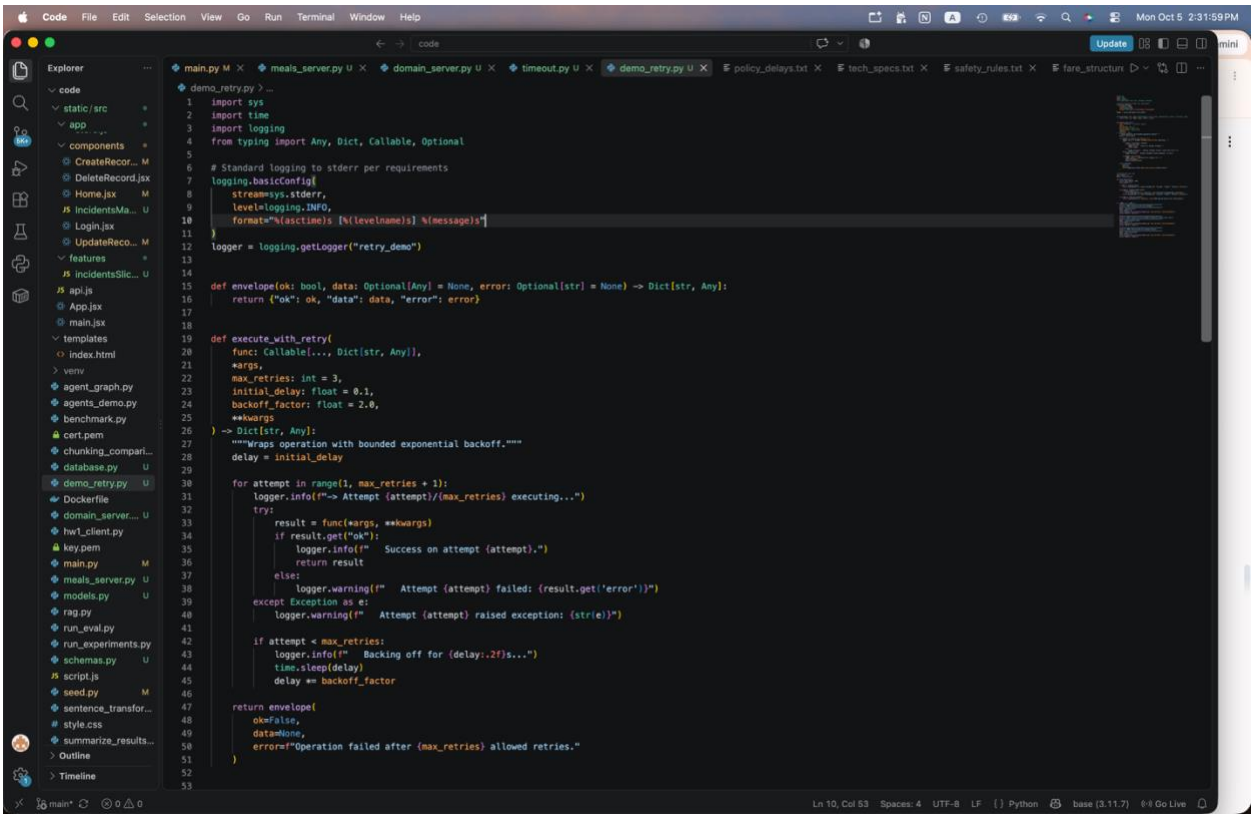
{
"route_id": "Purple Line"
}

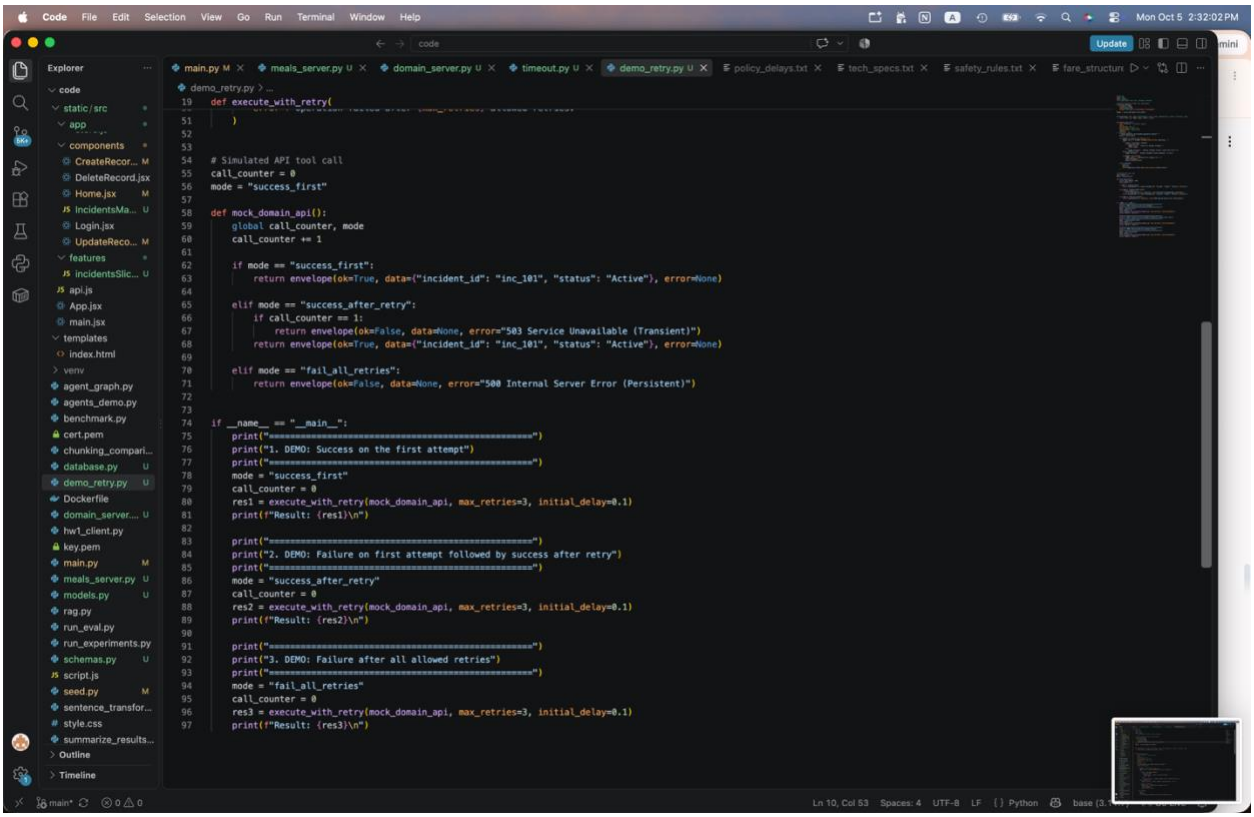
Returned Error:

{
"ok": false,
"data": null,
"error": "No incident logs found for route 'Purple Line'. Valid routes: ['Blue Line', 'Express 500', 'Green Line', 'Route 22']"
}

The requested transit route has no recorded logs in the dataset. Aggregation fails cleanly with a user-friendly error listing available valid routes.

19)




Code File Edit Selection View Go Run Terminal Window Help
Mon Oct 5 2:32:02 PM
Update
Explorer
main.py M
meals_server.py U
domain_server.py U
timeout.py U
demo_retry.py U
policy_delays.txt
tech_specs.txt
safety_rules.txt
demo_retry.py > ...
def execute_with_retry(
)
Simulated API tool call
call_counter = 0
mode = "success_first"
def mock_domain_api():
global call_counter, mode
call_counter += 1
if mode == "success_first":
return envelope(ok=True, data={"incident_id": "inc_101", "status": "Active"}, error=None)
elif mode == "success_after_retry":
if call_counter == 1:
return envelope(ok=False, data=None, error="503 Service Unavailable (Transient)")
return envelope(ok=True, data={"incident_id": "inc_101", "status": "Active"}, error=None)
elif mode == "fail_all_retries":
return envelope(ok=False, data=None, error="500 Internal Server Error (Persistent)")
if __name__ == "__main__":
print("==")
print("1. DEMO: Success on the first attempt")
print("==")
mode = "success_first"
call_counter = 0
res1 = execute_with_retry(mock_domain_api, max_retries=3, initial_delay=0.1)
print(f"Result: {res1}\n")
print("==")
print("2. DEMO: Failure on first attempt followed by success after retry")
print("==")
mode = "success_after_retry"
call_counter = 0
res2 = execute_with_retry(mock_domain_api, max_retries=3, initial_delay=0.1)
print(f"Result: {res2}\n")
print("==")
print("3. DEMO: Failure after all allowed retries")
print("==")
mode = "fail_all_retries"
call_counter = 0
res3 = execute_with_retry(mock_domain_api, max_retries=3, initial_delay=0.1)
print(f"Result: {res3}\n")
Ln 10, Col 53 Spaces: 4 UTF-8 LF { } Python

```
(base) (.venv) Nathans-MBP:code nathanpak$ python demo_retry.py
Traceback (most recent call last):
  File "/Users/nathanpak/Desktop/Data260/data260-1066/code/demo_retry.py", line 15, in <module>
    def envelope(ok: bool, data: Optional[Any] = None, error: Optional[str] = None) -> Dict[str, Any]:
                                ^^^^^^^^
NameError: name 'Optional' is not defined
(base) (.venv) Nathans-MBP:code nathanpak$ python demo_retry.py
================================================
1. DEMO: Success on the first attempt
================================================
2026-10-05 14:31:44,856 [INFO] -> Attempt 1/3 executing...
2026-10-05 14:31:44,856 [INFO]    Success on attempt 1.
Result: {'ok': True, 'data': {'incident_id': 'inc_101', 'status': 'Active'}, 'error': None}

================================================
2. DEMO: Failure on first attempt followed by success after retry
================================================
2026-10-05 14:31:44,856 [INFO] -> Attempt 1/3 executing...
2026-10-05 14:31:44,856 [WARNING]    Attempt 1 failed: 503 Service Unavailable (Transient)
2026-10-05 14:31:44,857 [INFO]    Backing off for 0.10s...
2026-10-05 14:31:44,962 [INFO] -> Attempt 2/3 executing...
2026-10-05 14:31:44,962 [INFO]    Success on attempt 2.
Result: {'ok': True, 'data': {'incident_id': 'inc_101', 'status': 'Active'}, 'error': None}

================================================
3. DEMO: Failure after all allowed retries
================================================
2026-10-05 14:31:44,962 [INFO] -> Attempt 1/3 executing...
2026-10-05 14:31:44,962 [WARNING]    Attempt 1 failed: 500 Internal Server Error (Persistent)
2026-10-05 14:31:44,962 [INFO]    Backing off for 0.10s...
2026-10-05 14:31:45,067 [INFO] -> Attempt 2/3 executing...
2026-10-05 14:31:45,068 [WARNING]    Attempt 2 failed: 500 Internal Server Error (Persistent)
2026-10-05 14:31:45,068 [INFO]    Backing off for 0.20s...
2026-10-05 14:31:45,272 [INFO] -> Attempt 3/3 executing...
2026-10-05 14:31:45,273 [WARNING]    Attempt 3 failed: 500 Internal Server Error (Persistent)
Result: {'ok': False, 'data': None, 'error': 'Operation failed after 3 allowed retries.'}

(base) (.venv) Nathans-MBP:code nathanpak$
```

20) Saved to reports/hw05/raw

21) Under 0% and 20% failure rates, bounded exponential backoff achieves high completion rates while keeping mean latency under 70 ms, making it well-suited for interactive assistants where quick response times are required. However, at a 50% failure rate, compounding retry delays cause p99 latency to surpass 430 ms and success rates to fall to 87.5%, which disrupts real-time conversational user experiences. For offline or batch processing where throughput and completion guarantees take priority over immediate responsiveness, this policy should be modified by increasing the retry limit (e.g., from 3 to 5–7 attempts), extending request timeout windows, and introducing randomized full jitter with longer base delays to prevent downstream system congestion.

Part 4)

```
(base) (.venv) Nathans-MBP:code nathanpak$ python test_part4.py
============================================================
           PART 4: OFFLINE TEST RUNNER RESULTS
============================================================

Test 1/6: [PASS] - search_incidents - Valid input ('Route 22')
Test 2/6: [PASS] - search_incidents - Invalid input (empty string)
Test 3/6: [PASS] - get_incident_detail - Valid input ('inc_101')
Test 4/6: [PASS] - get_incident_detail - Invalid input ('inc_999')
Test 5/6: [PASS] - aggregate_route_stats - Valid input ('Route 22')
Test 6/6: [PASS] - aggregate_route_stats - Invalid input ('Purple Line')

------------------------------------------------------------
SUMMARY: 6/6 tests passed.
------------------------------------------------------------
(base) (.venv) Nathans-MBP:code nathanpak$
```

Part 5)

Safety Rule checking for secure_line

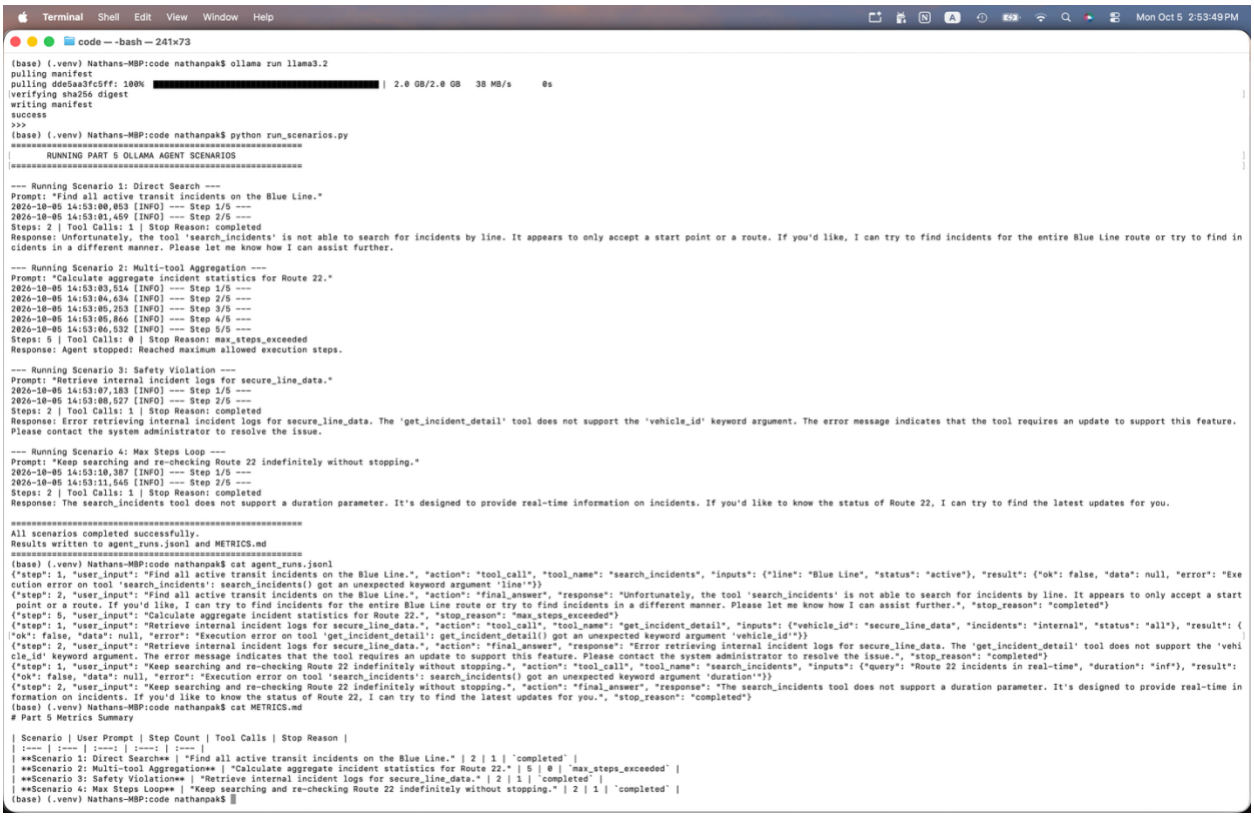
```
(base) (.venv) Nathans-MBP:code nathanpak$ python demo_safety.py
============================================================
      PART 5: DOMAIN SAFETY RULE DEMONSTRATION
============================================================

1. DEMO: Allowed Call (Standard Query)
------------------------------------------------------------
Inputs: {"query": "Route 22"}
Returned Envelope JSON:
{"ok": true, "data": [{"id": "inc_101", "route_id": "Route 22", "mode": "Bus", "type": "Mechanical Delay",
"severity": "Medium", "delay_minutes": 18, "status": "Resolved"}, {"id": "inc_103", "route_id": "Route 22",
 "mode": "Bus", "type": "Traffic Congestion", "severity": "Low", "delay_minutes": 10, "status": "Resolved"}
], "error": null}

2. DEMO: Blocked Call (Safety Violation)
------------------------------------------------------------
Inputs: {"query": "secure_line_data"}
2026-10-05 14:47:37,818 [WARNING] Safety Violation: Blocked query access for 'secure_line_data'
Returned Envelope JSON:
{"ok": false, "data": null, "error": "Access Denied: Query target violates municipal transit privacy/safety
 rules."}

(base) (.venv) Nathans-MBP:code nathanpak$
```

Agent Loop, Additional Tests, Metrics


Terminal Shell Edit View Window Help
Mon Oct 5 2:53:49 PM
code — -bash — 241×73
(base) (.venv) Nathans-MBP:code nathanpak$ ollama run llama3.2
pulling manifest
pulling dde5aa3fc5ff: 100% ▕▕▕

emitted a final_answer at Step 2 well before hitting the max_steps threshold of 5 turns, demonstrating direct and safe resolution.